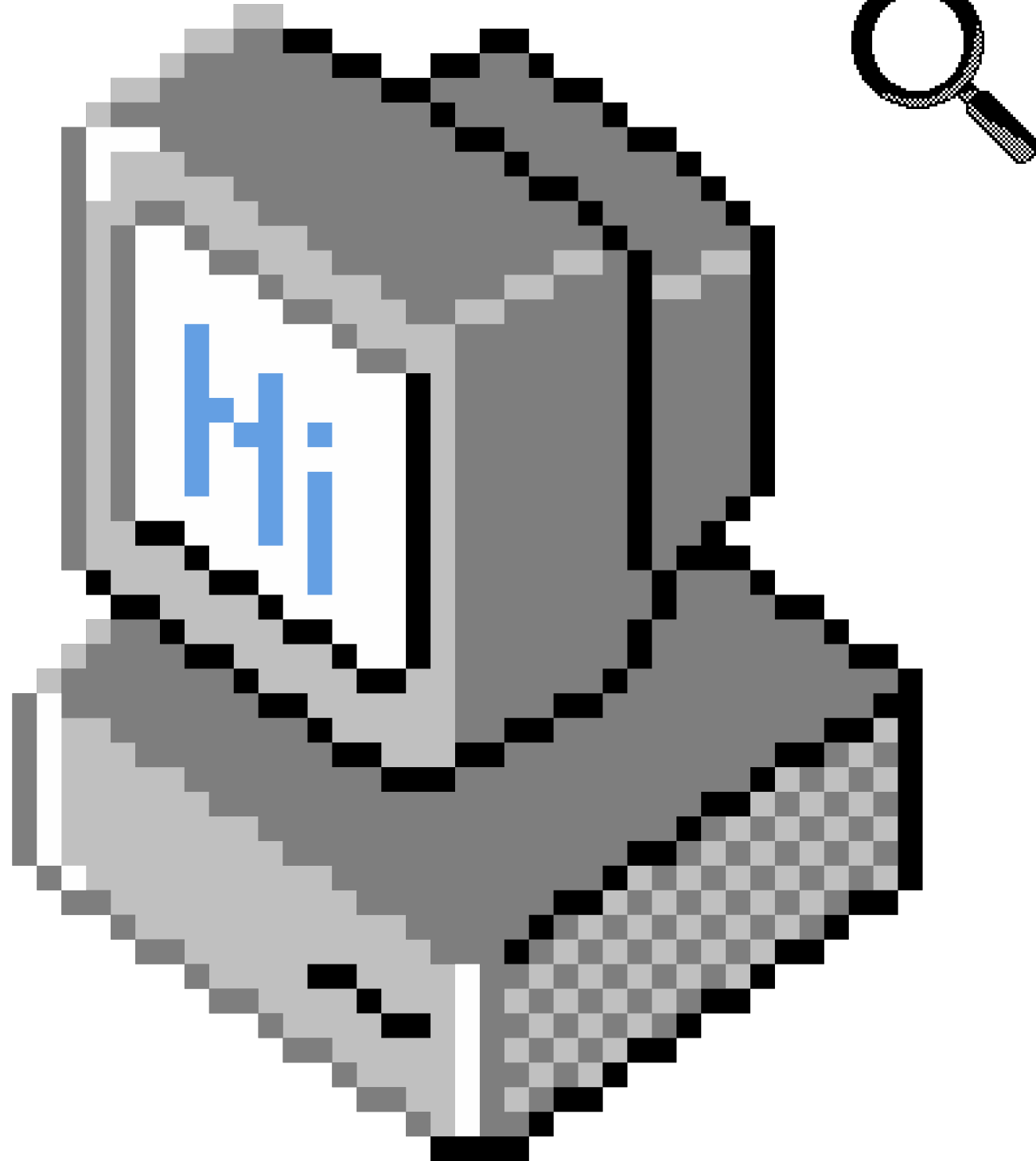


PBL5:DỰ ÁN KÌ THUẬT MÁY TÍNH

Báo cáo tiến độ hàng tuần

Group 01



Presented by Olivia Wilson

TỐI ƯU LƯỒNG VIDEO ĐẦU VÀO (INPUT PIPELINE)

Page 02



- Webcam Logitech C270 là camera chuẩn HD (1280x720)
- khi gọi hàm `cv2.VideoCapture(0)` cơ bản, thư viện OpenCV thường tự động ép độ phân giải xuống mức thấp nhất (thường là 640x480) để chạy cho nhẹ. Phóng to một cái ảnh 640x480 lên toàn màn hình thì chắc chắn sẽ bị rỗ và mờ.
- Webcam USB (như C270): Dữ liệu hình ảnh phải đi qua cổng USB, bị nén lại, sau đó CPU của Jetson phải giải mã (decode) luồng dữ liệu đó thành hình ảnh. Quá trình này rất mất thời gian và tốn tài nguyên.

- Camera CSI: Cổng CSI kết nối trực tiếp vào Bộ xử lý tín hiệu hình ảnh (ISP) của lõi Jetson Nano. Dữ liệu đi thẳng vào "não" mà không cần qua khâu trung gian nào, giúp giảm độ trễ gần như bằng 0 và giải phóng hoàn toàn gánh nặng cho CPU.

```
import cv2

# Mở luồng camera USB (số 0 thường là camera mặc định)
cap = cv2.VideoCapture(0)

print("Đang mở Camera... Nhấn phím 'q' trên cửa sổ video để thoát.")

while True:
    ret, frame = cap.read()
    if not ret:
        print("Không thể nhận luồng video!")
        break

    # Hiển thị khung hình lên màn hình GUI
    cv2.imshow("Khoa PBL5 - Camera Test", frame)

    # Thoát nếu nhấn phím 'q'
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

```
import cv2

cap = cv2.VideoCapture(0)

# Chèn thêm 2 dòng này để ép lên HD 720p
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 1280)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 720)

print("Đang mở Camera... Nhấn phím 'q' trên cửa sổ video để thoát.")
# ... (phần code bên dưới giữ nguyên)
```

TỐI ƯU LƯỒNG VIDEO ĐẦU VÀO (INPUT PIPELINE)

-Tất cả các chế độ phần cứng mà chiếc camera Logitech C270 hỗ trợ. Nó chia làm 2 định dạng chính ([0] và [1]):

+Định dạng [0]: 'YUYV' (Video thô, không nén)

Ý nghĩa: Nếu lấy video HD ở dạng thô không nén, do băng thông cáp USB 2.0 quá hẹp, camera bị nghẽn và chỉ có thể xuất ra tối đa 10 khung hình/giây (10 FPS).

+Định dạng [1]: 'MJPG' (Video nén Motion-JPEG)

Ý nghĩa: Bằng cách nén hình ảnh lại ngay trên phần cứng của camera (thành định dạng MJPG) trước khi truyền qua dây USB, camera C270 hoàn toàn đạt được chuẩn HD 720p ở tốc độ 30 FPS mượt mà.

[0]: 'YUYV' (YUYV 4:2:2)

Size: Discrete 1280x720

Interval: Discrete 0.100s (10.000 fps)

Interval: Discrete 0.133s (7.500 fps)

Interval: Discrete 0.200s (5.000 fps)

'MJPG' (Motion-JPEG, compressed)

Size: Discrete 1280x720

Interval: Discrete 0.033s (30.000 fps)

Interval: Discrete 0.042s (24.000 fps)

Interval: Discrete 0.050s (20.000 fps)

Interval: Discrete 0.067s (15.000 fps)

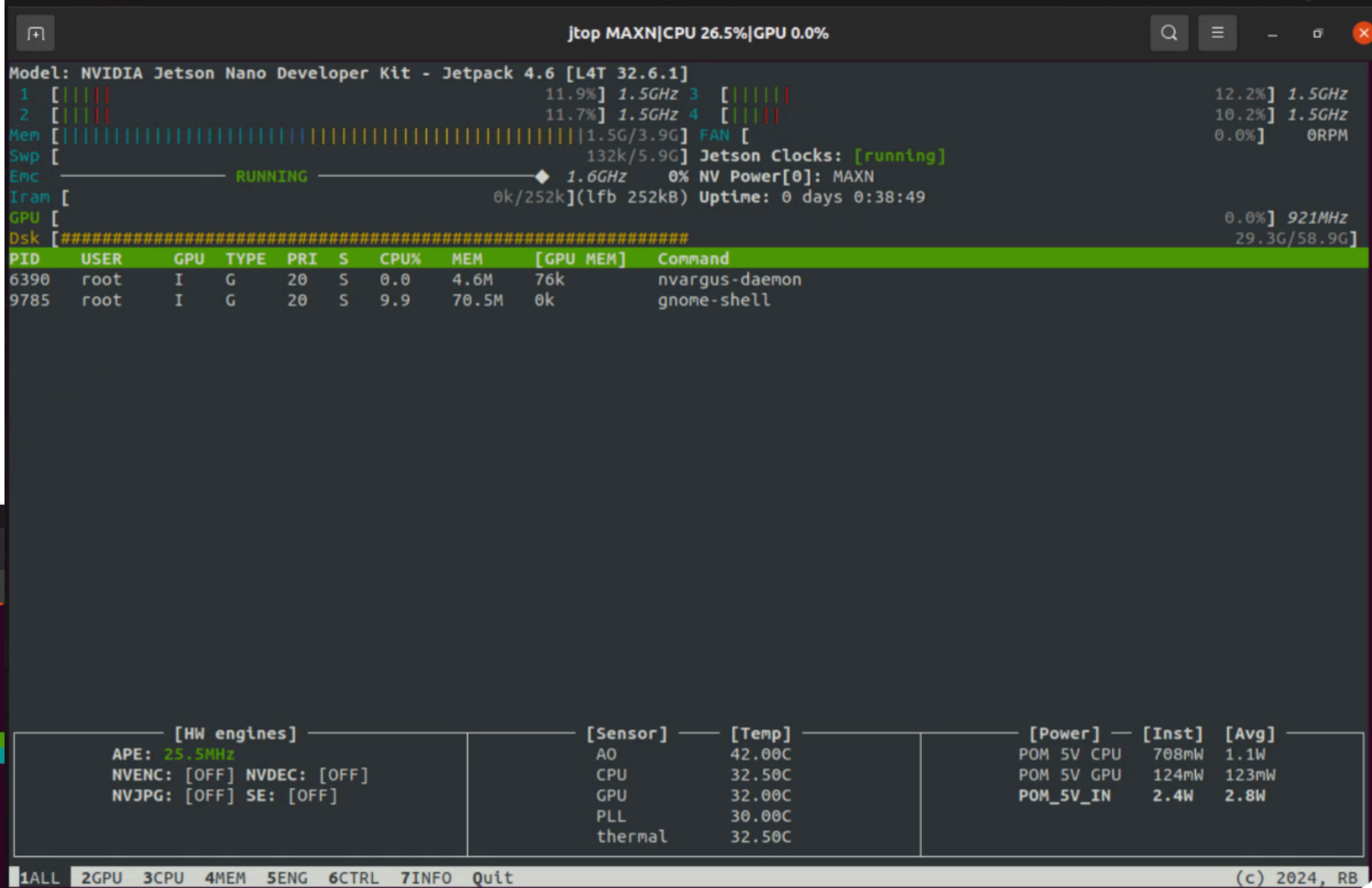
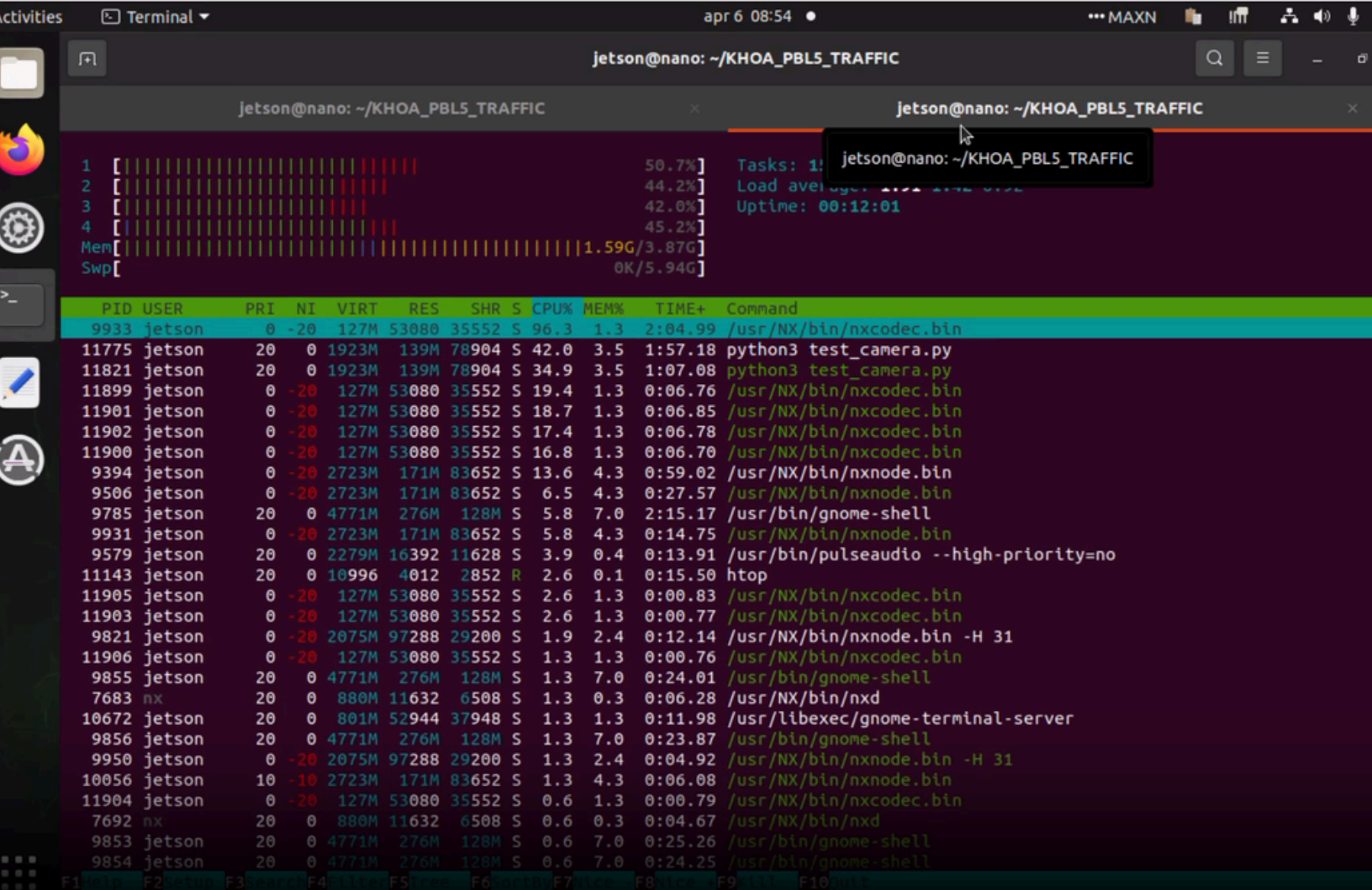
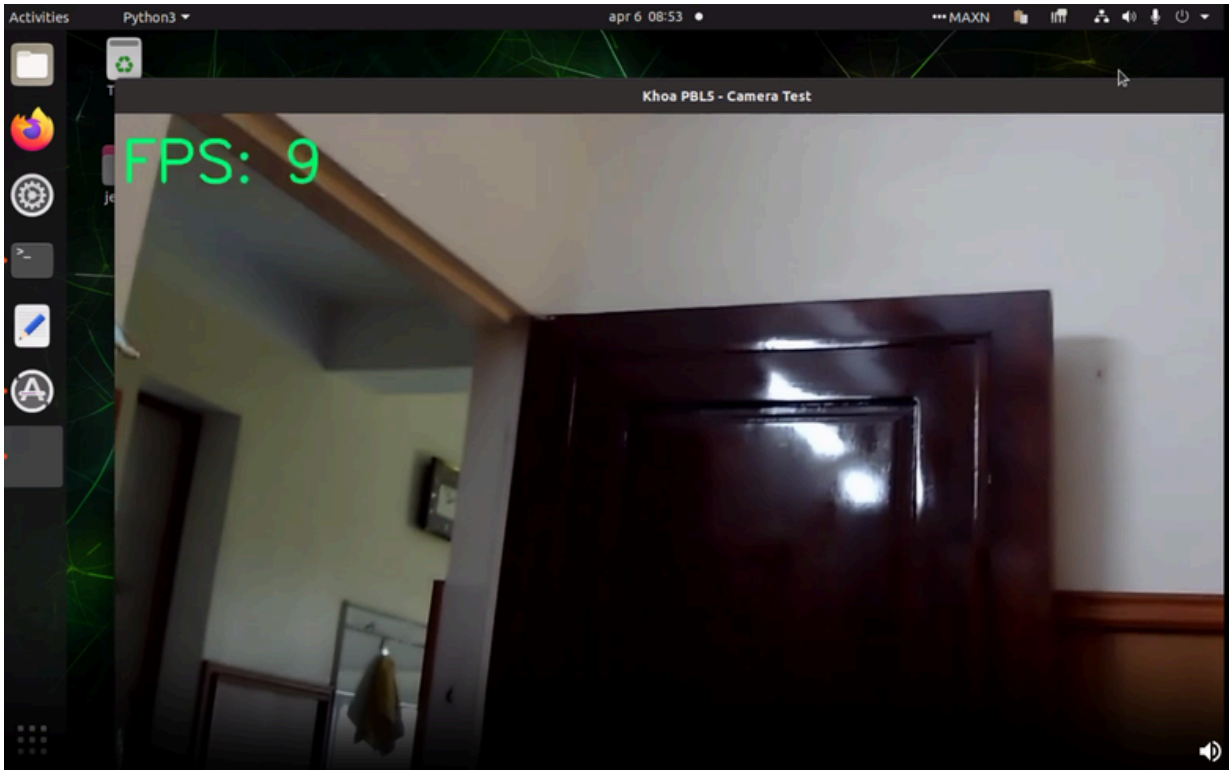
Interval: Discrete 0.100s (10.000 fps)

Interval: Discrete 0.133s (7.500 fps)

Interval: Discrete 0.200s (5.000 fps)



TỐI ƯU LUỒNG VIDEO ĐẦU VÀO (INPUT PIPELINE)

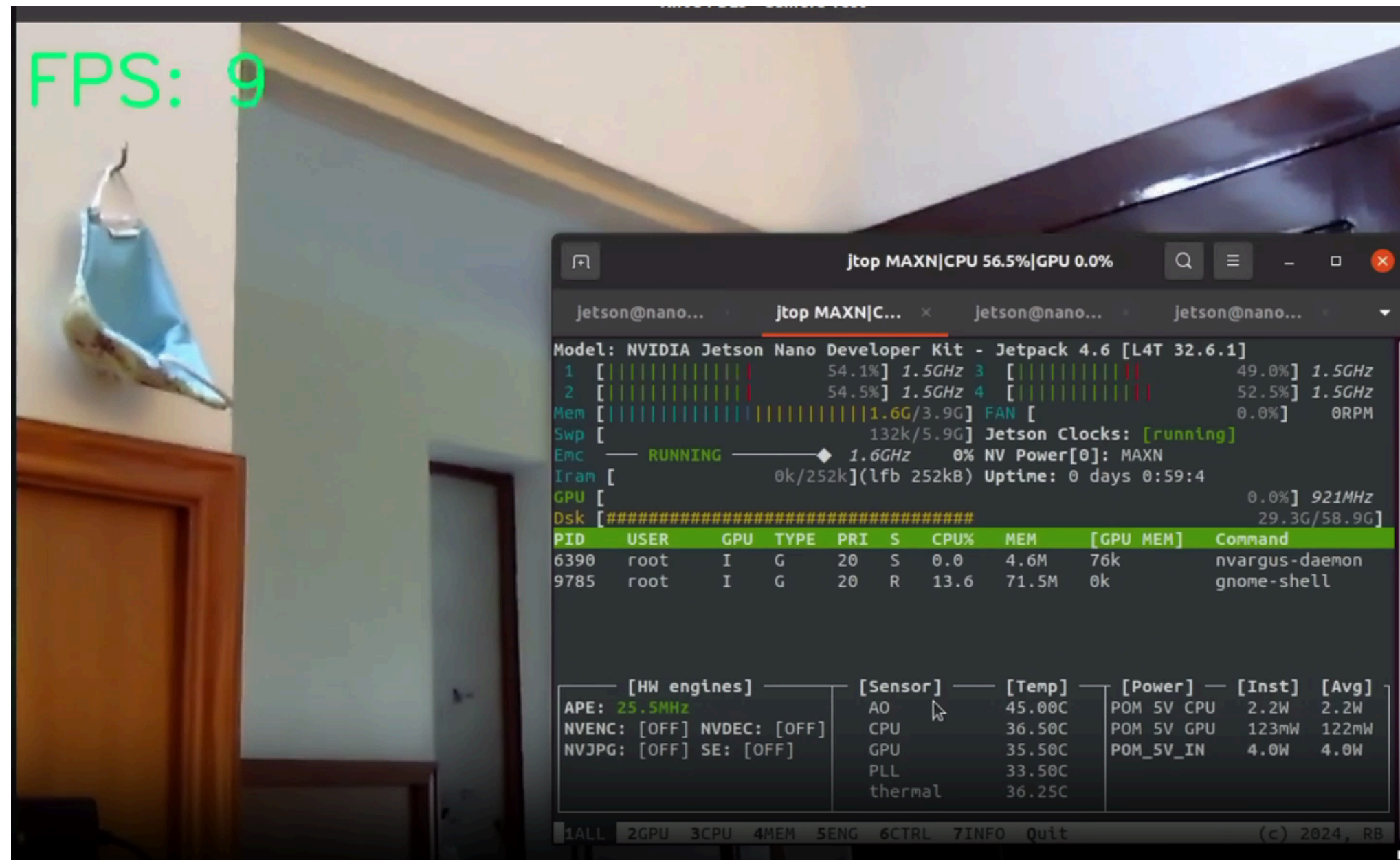


```
# Ép dùng định dạng MJPG (Motion-JPEG)
cap.set(cv2.CAP_PROP_FOURCC, cv2.VideoWriter_fourcc(*'MJPG'))
```

v4l2-ctl

TỐI ƯU LƯỒNG VIDEO ĐẦU VÀO (INPUT PIPELINE)

FPS: 9

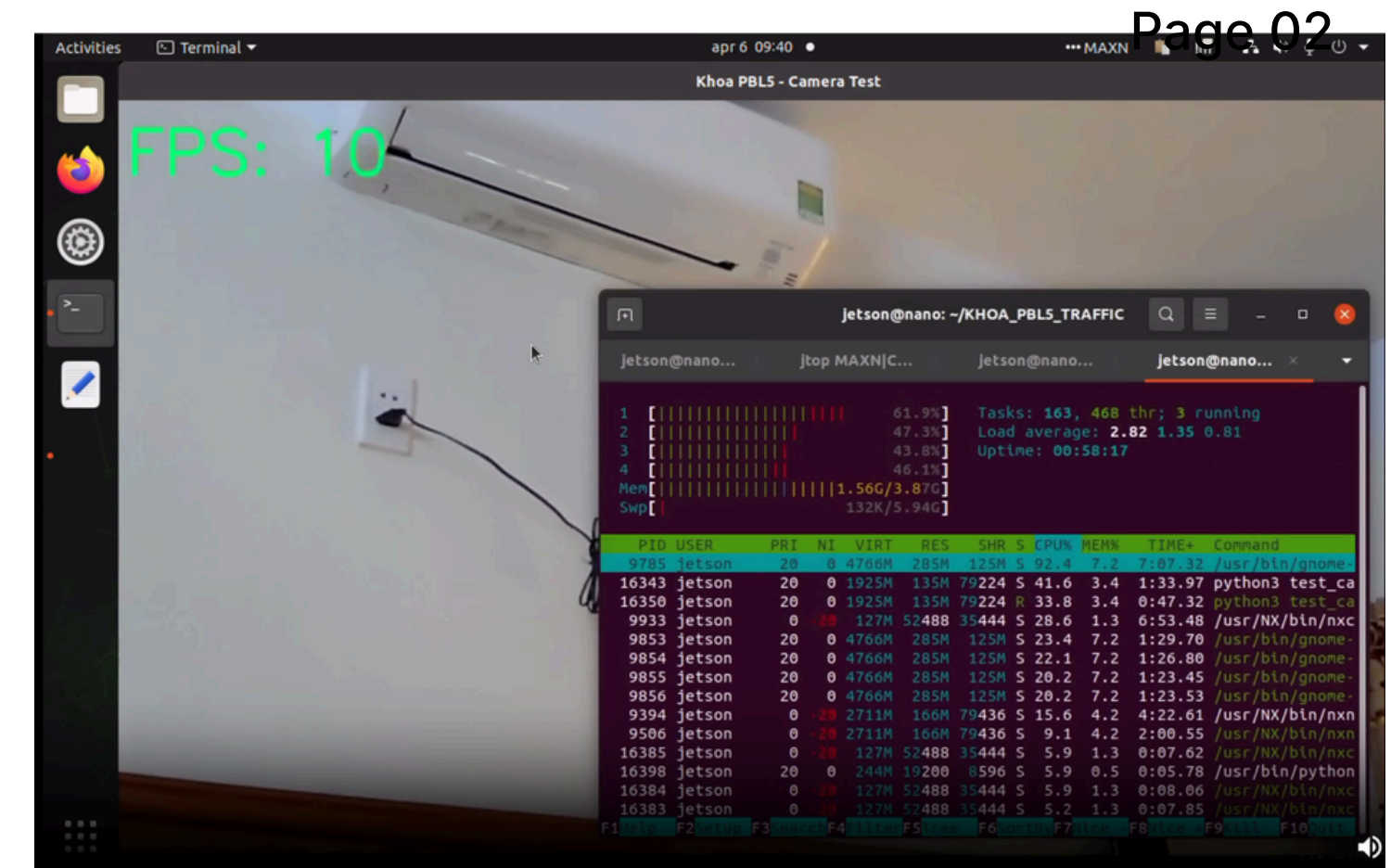


```
jetson@nano... jtop MAXN|CPU 56.5%|GPU 0.0%  
Model: NVIDIA Jetson Nano Developer Kit - Jetpack 4.6 [L4T 32.6.1]  
1 [|||||] 54.1% 1.5GHz 3 [|||||] 49.0% 1.5GHz  
2 [|||||] 54.5% 1.5GHz 4 [|||||] 52.5% 1.5GHz  
Mem [|||||] 1.6G/3.9G FAN [|||||] 0.0% 0RPM  
Swp [|||||] 132k/5.9G Jetson Clocks: [running]  
Emc [|||||] 1.6GHz 0% NV Power[0]: MAXN  
Iram [|||||] 0k/252k(lfb 252kB) Uptime: 0 days 0:59:4  
GPU [|||||] 0.0% 921MHz  
Dsk [|||||] 29.3G/58.9G  
PID USER GPU TYPE PRI S CPU% MEM [GPU MEM] Command  
6390 root I G 20 S 0.0 4.6M 76k nvargus-daemon  
9785 root I G 20 R 13.6 71.5M 0k gnome-shell  
[HW engines] [Sensor] [Temp] [Power] [Inst] [Avg]  
APE: 25.5MHz AO 45.00C POM 5V CPU 2.2W 2.2W  
NVENC: [OFF] NVDEC: [OFF] CPU 36.50C POM 5V GPU 123mW 122mW  
NVJPG: [OFF] SE: [OFF] GPU 35.50C POM_5V_IN 4.0W 4.0W  
PLL 33.50C  
thermal 36.25C  
1ALL 2GPU 3CPU 4MEM 5ENG 6CTRL 7INFO Quit (c) 2024, RB
```

Page 02

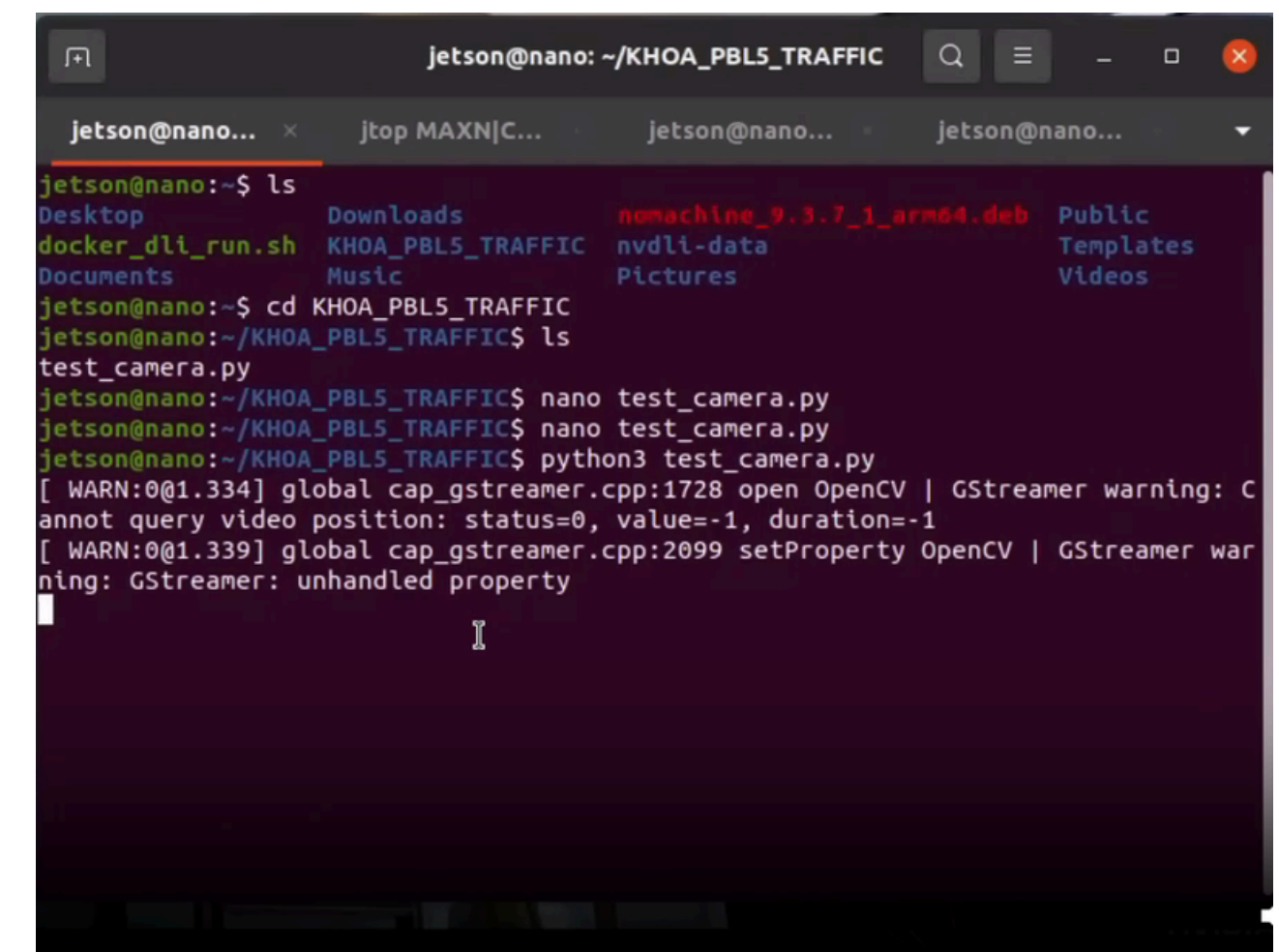
Khoa PBL5 - Camera Test

FPS: 10



```
jetson@nano: ~/KHOA_PBL5_TRAFFIC  
jetson@nano... jtop MAXN|C... jetson@nano... jetson@nano...  
1 [|||||] 61.9% Tasks: 163, 468 thr; 3 running  
2 [|||||] 47.3% Load average: 2.82 1.35 0.81  
3 [|||||] 43.8% Uptime: 00:58:17  
4 [|||||] 46.1%  
Mem [|||||] 1.56G/3.87G  
Swp [|||||] 132K/5.94G  
PID USER PRI NI VIRT RES SHR S CPU% MEM% TIME+ Command  
9785 jetson 20 0 4746M 285M 125M S 92.4 7.2 7:07.32 /usr/bin/gnome-  
16343 jetson 20 0 1925M 135M 79224 S 41.6 3.4 1:33.97 python3 test_ca  
16350 jetson 20 0 1925M 135M 79224 R 33.8 3.4 0:47.32 python3 test_ca  
9933 jetson 0 -28 127M 52488 35444 S 28.6 1.3 6:53.48 /usr/NX/bin/nxc  
9853 jetson 20 0 4766M 285M 125M S 23.4 7.2 1:29.70 /usr/bin/gnome-  
9854 jetson 20 0 4766M 285M 125M S 22.1 7.2 1:26.80 /usr/bin/gnome-  
9855 jetson 20 0 4766M 285M 125M S 20.2 7.2 1:23.45 /usr/bin/gnome-  
9856 jetson 20 0 4766M 285M 125M S 20.2 7.2 1:23.53 /usr/bin/gnome-  
9394 jetson 0 -28 2711M 166M 79436 S 15.6 4.2 4:22.61 /usr/NX/bin/nxn  
9506 jetson 0 -28 2711M 166M 79436 S 9.1 4.2 2:00.55 /usr/NX/bin/nxn  
16385 jetson 0 -28 127M 52488 35444 S 5.9 1.3 0:07.62 /usr/NX/bin/nxc  
16398 jetson 20 0 244M 19200 8596 S 5.9 0.5 0:05.78 /usr/bin/python  
16384 jetson 0 -28 127M 52488 35444 S 5.9 1.3 0:08.06 /usr/NX/bin/nxc  
16383 jetson 0 -28 127M 52488 35444 S 5.2 1.3 0:07.85 /usr/NX/bin/nxc  
F1 F2 F3 F4 F5 F6 F7 F8 F9 F10
```

jetson@nano: ~/KHOA_PBL5_TRAFFIC



```
jetson@nano:~$ ls  
Desktop Downloads nomachine_9.3.7_1_arm64.deb Public  
docker_dli_run.sh KHOA_PBL5_TRAFFIC nvdli-data Templates  
Documents Music Pictures Videos  
jetson@nano:~$ cd KHOA_PBL5_TRAFFIC  
jetson@nano:~/KHOA_PBL5_TRAFFIC$ ls  
test_camera.py  
jetson@nano:~/KHOA_PBL5_TRAFFIC$ nano test_camera.py  
jetson@nano:~/KHOA_PBL5_TRAFFIC$ nano test_camera.py  
jetson@nano:~/KHOA_PBL5_TRAFFIC$ python3 test_camera.py  
[ WARN:0@1.334] global cap_gstreamer.cpp:1728 open OpenCV | GStreamer warning: C  
annot query video position: status=0, value=-1, duration=-1  
[ WARN:0@1.339] global cap_gstreamer.cpp:2099 setProperty OpenCV | GStreamer war  
ning: GStreamer: unhandled property
```

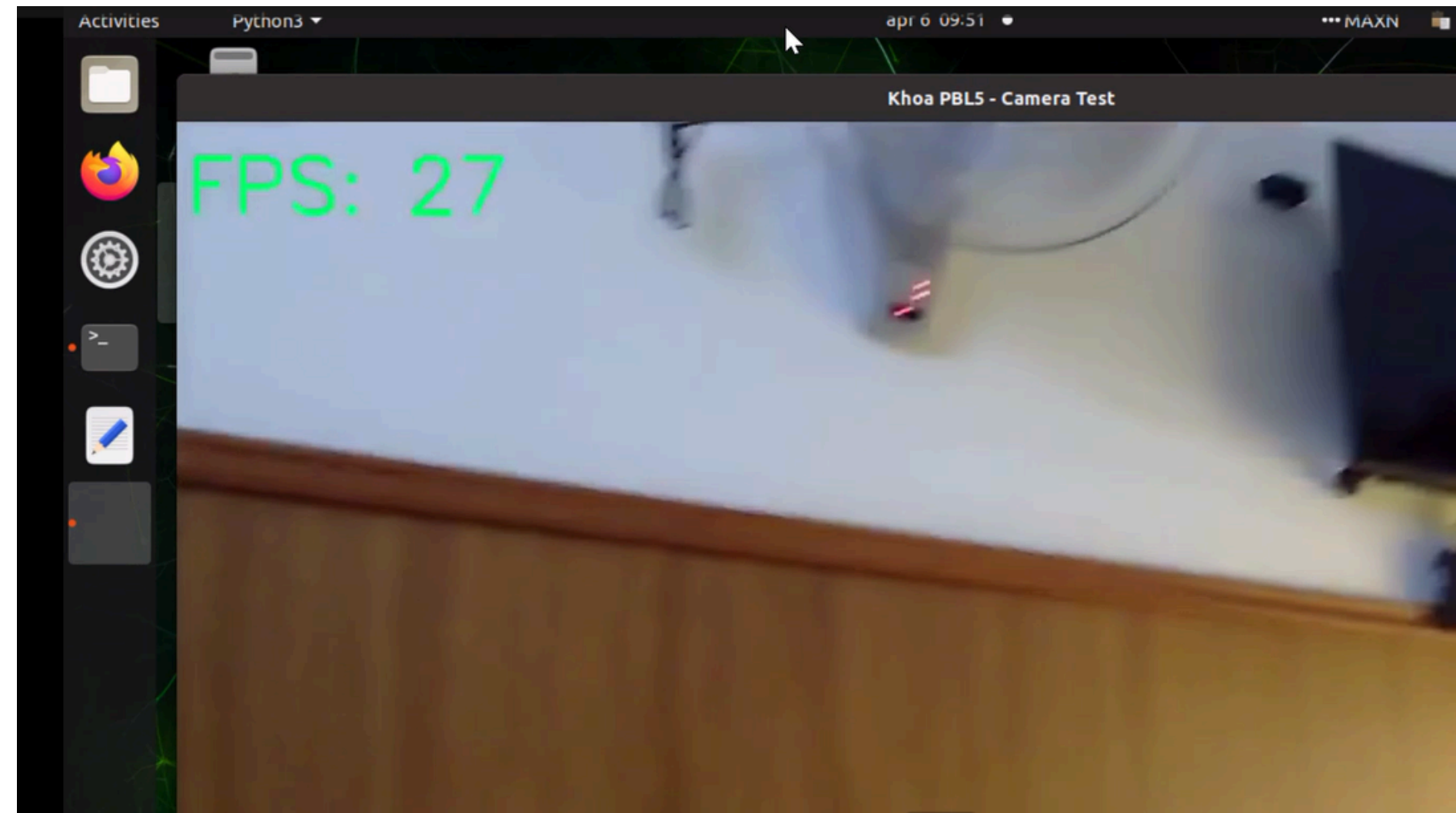
TỐI ƯU LƯỒNG VIDEO ĐẦU VÀO (INPUT PIPELINE)

Page 02

```
jetson@nano:~/KHOA_PBL5_TRAFFIC$ python3 test_camera.py
jetson@nano:~/KHOA_PBL5_TRAFFIC$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 20.04.6 LTS
Release:        20.04
Codename:       focal
jetson@nano:~/KHOA_PBL5_TRAFFIC$ python3 --version
Python 3.8.10
```

-Hệ thống phần mềm của bản
20.04 không hiểu được trình điều
khiển camera phần cứng của bản
18.04

```
#-Ngăn openCV sử dụng phần mềm trung gian GStreamer, và ép nó giao tiếp thẳng với  
# driver lõi của Camera (gọi là Video4Linux2 - viết tắt là V4L2).  
cap = cv2.VideoCapture(0, cv2.CAP_V4L2)
```



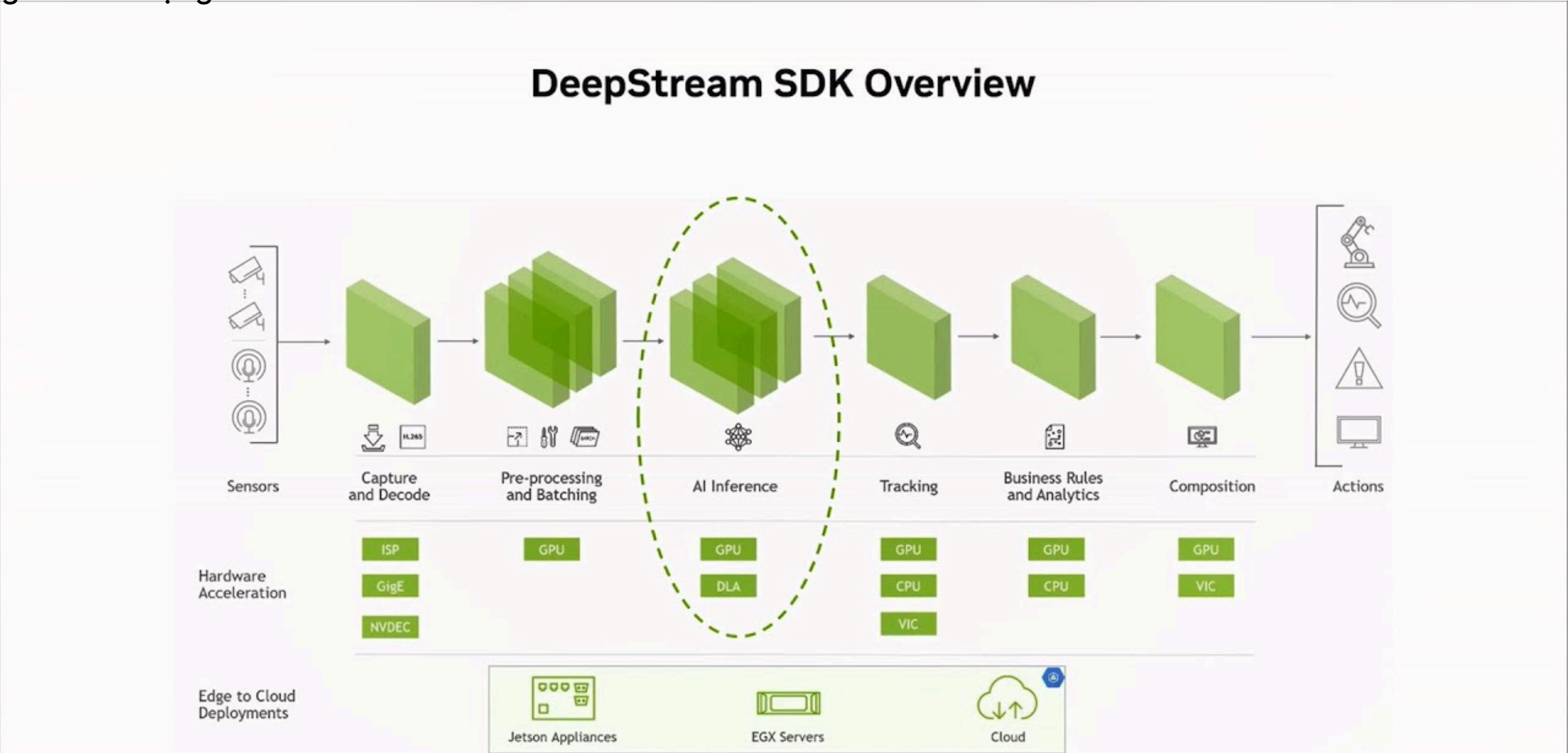
ĐỀ XUẤT LỘ TRÌNH TỐI ƯU

Dựa trên các lỗi đã khảo sát, nhóm đề xuất tái cấu trúc hệ thống theo 3 bước:

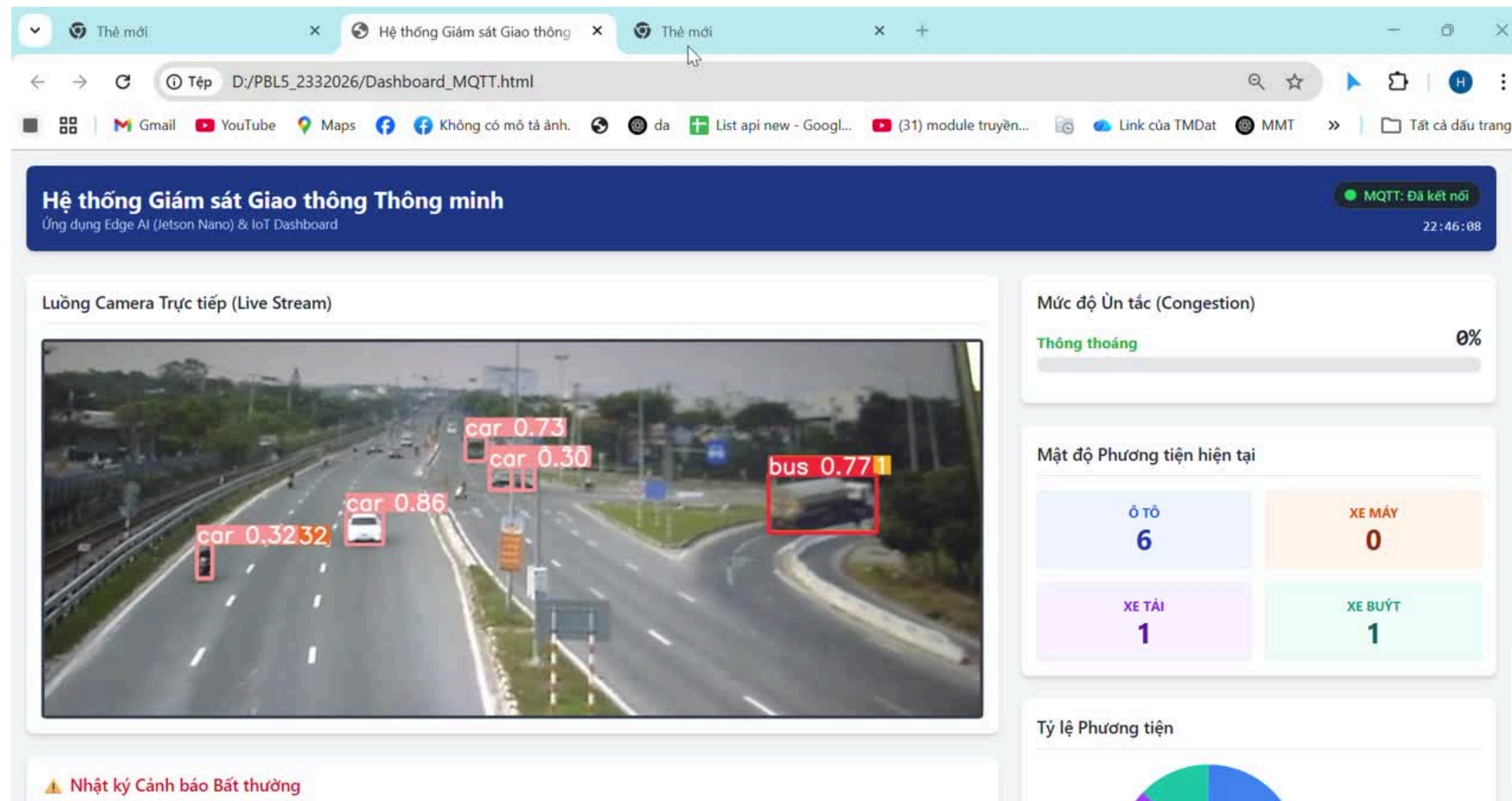
- Flash lại Firmware: Cài đặt lại hệ thống về nguyên bản JetPack 4.6.1 (Ubuntu 18.04) để khôi phục sự ổn định của Lỗi Driver GPU.
- Cô lập Môi trường: Triển khai Docker Container chứa Python 3.8+ và PyTorch để chạy các mô hình AI hiện đại mà không làm hỏng OS gốc.
- Triển khai DeepStream SDK: Thay thế hoàn toàn OpenCV. Sử dụng DeepStream để đẩy video thẳng vào bộ giải mã phần cứng NVDEC và áp dụng cơ chế Zero-Memory-Copy, giúp dữ liệu nằm trên GPU, giải phóng sức lao động cho CPU.



- nvdec
- nvstreammux
- nvinfer
- nvtracker
- nvdsanalytics



IOT WEB DASHBOARD



Page 02

-Thách thức: Giám sát giao thông yêu cầu cập nhật liên tục (Real-time) nhưng băng thông mạng lại có giới hạn. Không thể truyền toàn bộ Video phân tích lên Cloud.

-Giải pháp - Kiến trúc Luồng kép độc lập:

+Kênh Media (HTTP/Flask): Jetson đóng vai trò là Web Server cục bộ (Localhost).

Truyền trực tiếp các khung hình (MJPEG) đã vẽ Bounding Box đến trình duyệt người dùng với độ trễ tối thiểu.

+Kênh Telemetry (MQTT): Jetson đóng gói các số liệu thống kê thành thông điệp văn bản siêu nhẹ (JSON) và gửi lên đám mây (HiveMQ Broker).

- Chuyển hóa tri thức (Serialization): Mảng ID phương tiện từ YOLOv8 được thuật toán Python đếm và bọc lại thành một tệp JSON có cấu trúc.
- Phát sóng (Publish): Sử dụng thư viện paho.mqtt, thiết bị biên đẩy gói JSON này lên tần số khoa_pbl5/traffic_data trên Broker.
- Vượt rào cản bảo mật Web: Trình duyệt sử dụng Paho MQTT Client kết nối vào cổng 8884 (WSS - WebSockets Secure) thay vì cổng 1883 truyền thống.
- Cập nhật DOM & Biểu đồ (Subscribe): Bắt sự kiện onMessageArrived, giải mã JSON và thao tác trực tiếp lên DOM (thay đổi con số) và Chart.js mà không cần tải lại trang.



```
# Đóng gói dữ liệu thành JSON siêu nhẹ
payload = {
    "counts": {"car": car_count, "moto": moto_count, "truck": truck_count, "bus": bus_count},
    "congestion": 0,
    "alert": "none"
}
# Đẩy lên Cloud Broker (Kích hoạt luồng chạy ngầm client.loop_start())
client.publish("khoa_pbl5/traffic_data", json.dumps(payload))
```

```
// Thiết lập kết nối bảo mật WSS để Web không bị chặn
mqtt_client.connect({ useSSL: true, onSuccess: onConnect });

// Bóc tách JSON và cập nhật Giao diện ngay lập tức
function onMessageArrived(message) {
    const data = JSON.parse(message.payloadString);
    document.getElementById('countCar').innerText = data.counts.car;
    // ... Cập nhật Chart.js và Thanh cảnh báo Ùn tắc ...
}
```

TRACKING ALGORITHMS

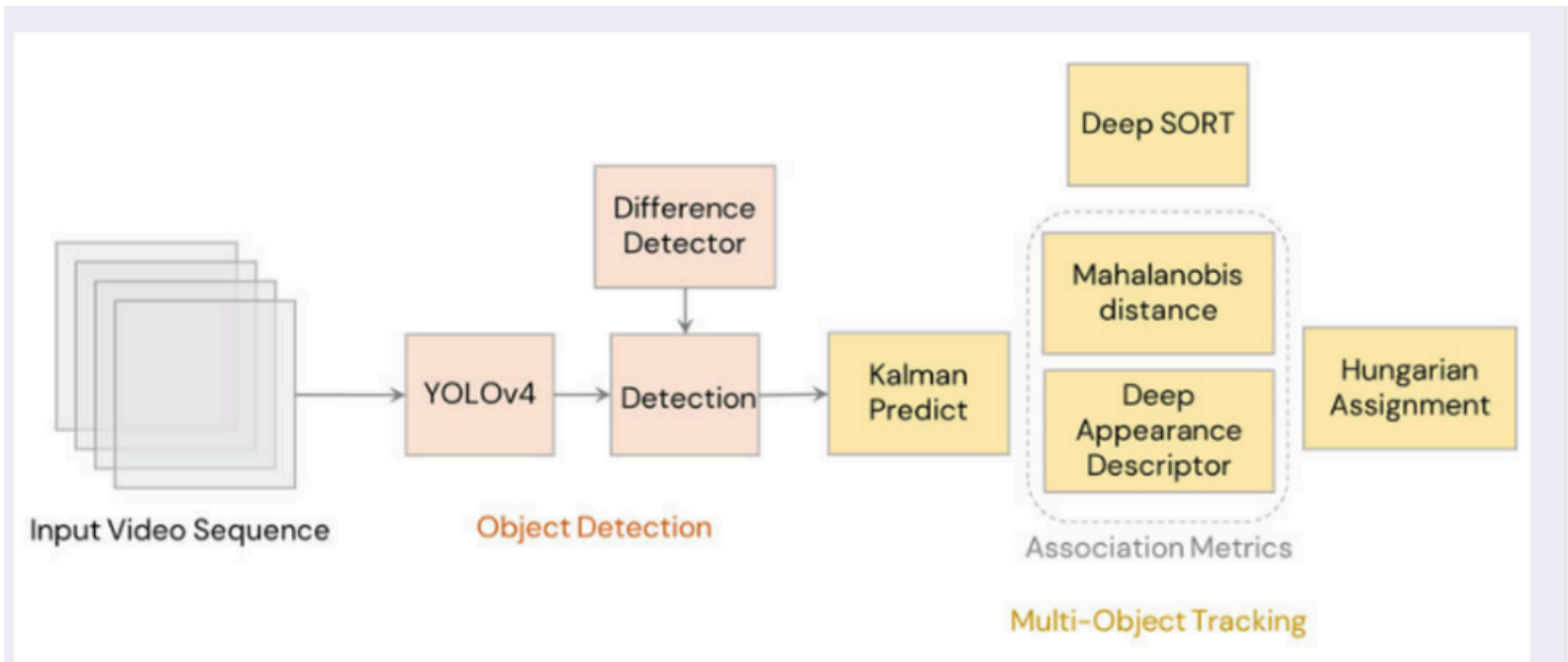


Figure 3: [DeepSORT Algorithm](#)

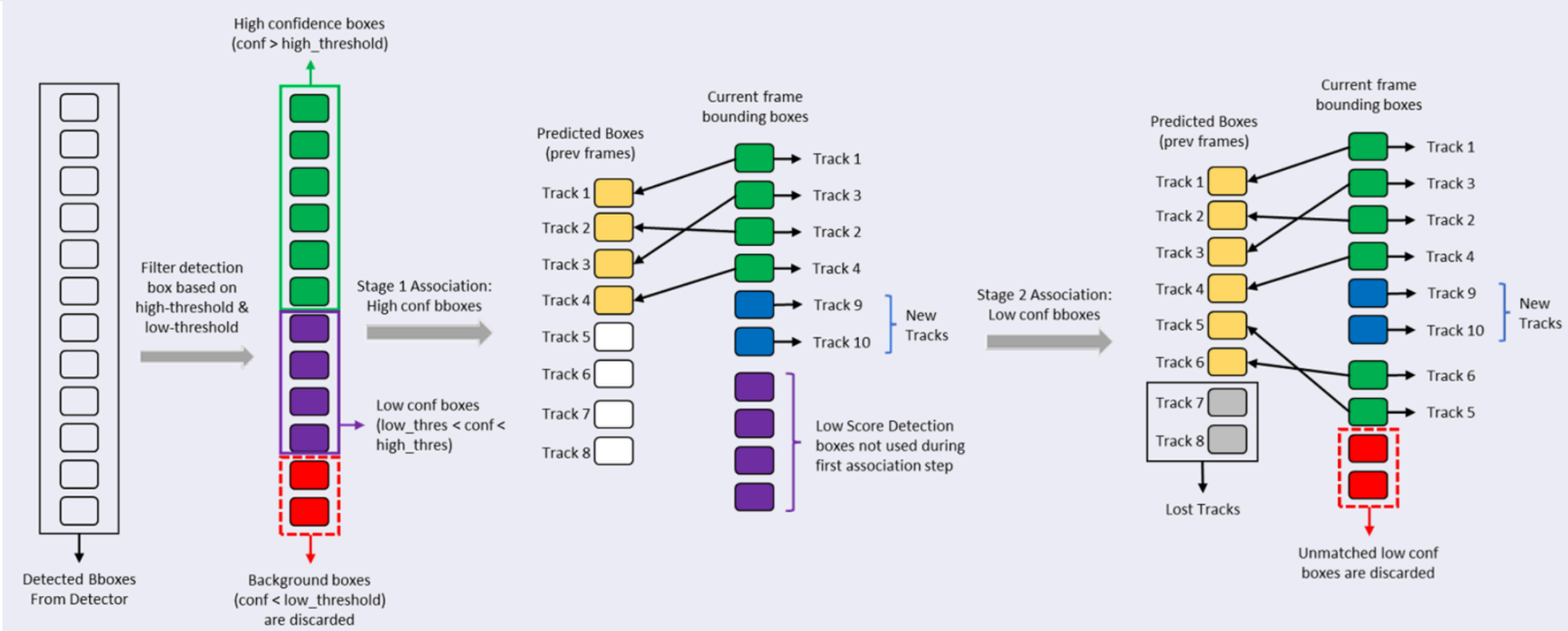


Figure 7: BYTETrack Algorithm

MẬT ĐỘ KHÔNG GIAN THÔ (ABSOLUTE COUNT)

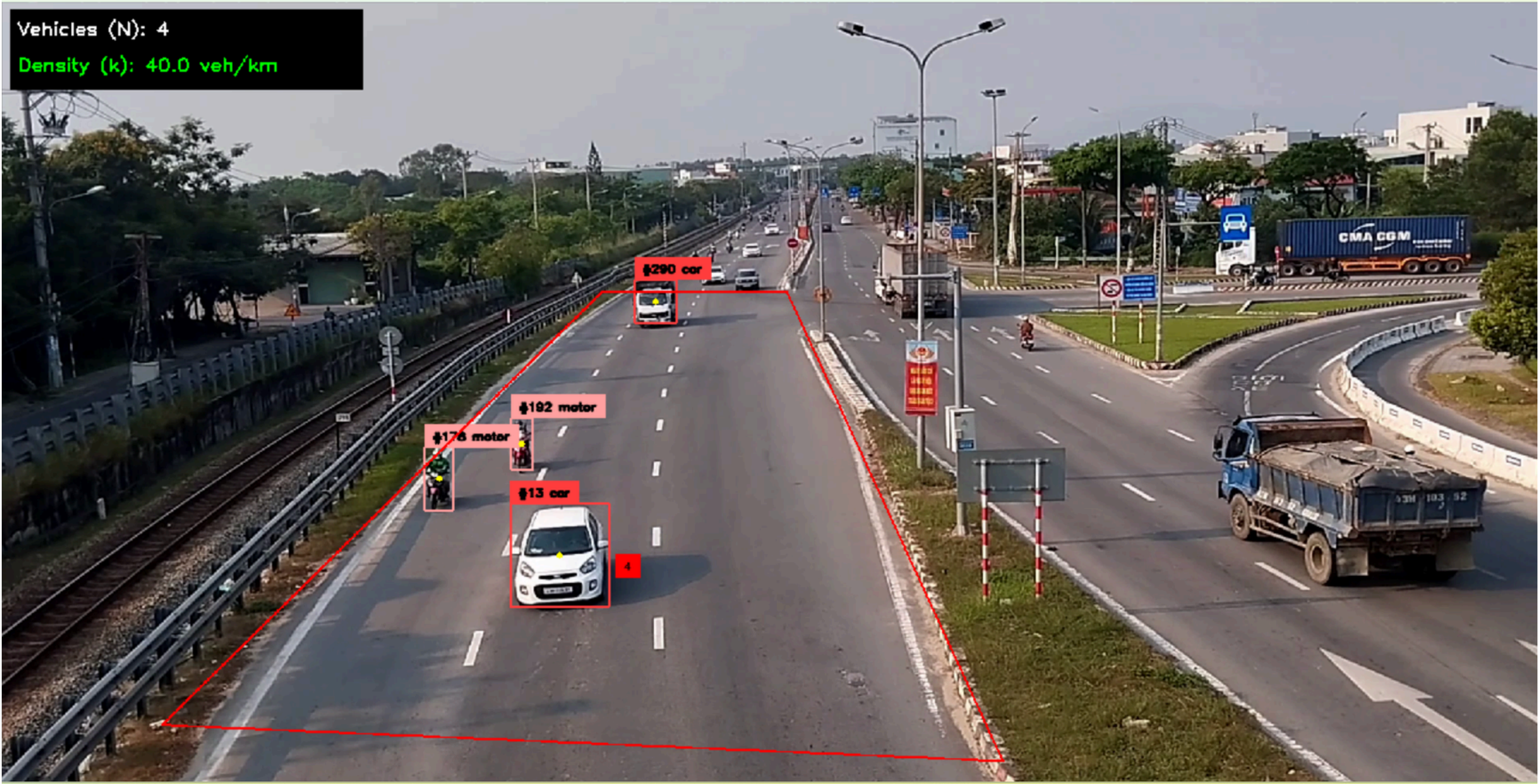
- Công thức:

$$k = \frac{N}{L}$$

(Trong đó: k là mật độ, N là tổng số xe đếm được trong vùng, L là chiều dài đoạn đường tính bằng km).

-Cách làm: định nghĩa một vùng đa giác (Region of Interest - ROI) trên camera. Ở mỗi khung hình, đếm xem có bao nhiêu Bounding Box đang nằm trong đó.

-Điểm yếu: Nó đánh đồng mọi loại xe. 20 chiếc xe máy sẽ được đánh giá ngang với 20 chiếc xe container, dẫn đến việc đánh giá sai hoàn toàn mức độ kẹt xe thực tế.



MẬT ĐỘ QUY ĐỔI TẢI TRỌNG (PCE-AWARE DENSITY)

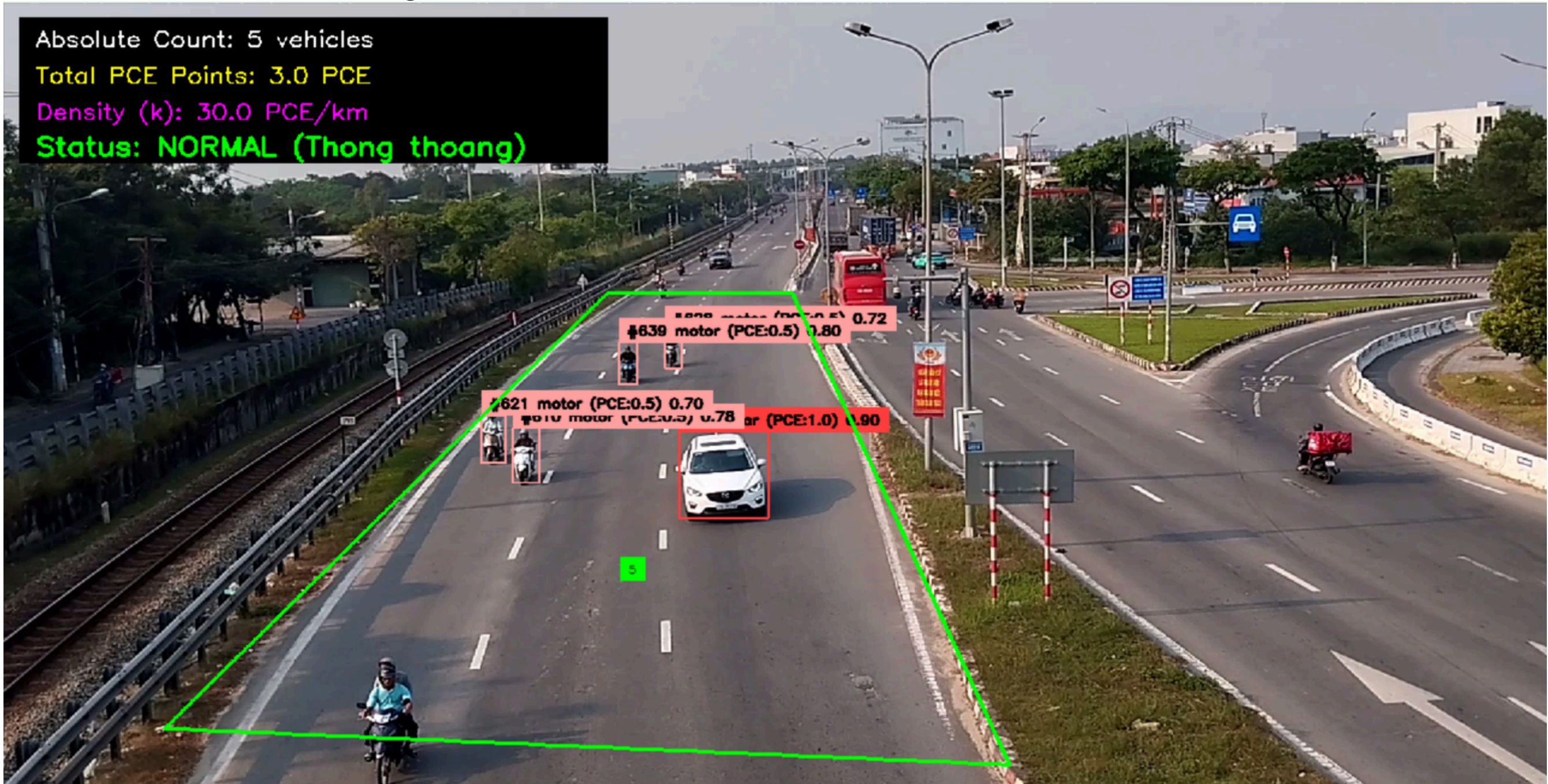
Công thức tính mật độ:

$$k = \frac{\sum_{i=1}^c (n_i \times \text{PCE}_i)}{A}$$

Trong đó:

n_i là số lượng xe của loại xe i ,
 PCE_i là hệ số quy đổi tương ứng
 A là chiều dài đoạn đường

```
PCE_WEIGHTS = {  
    "motor": 0.5,  
    "car": 1.0,  
    "bus": 2.5,  
    "truck": 2.5  
}
```



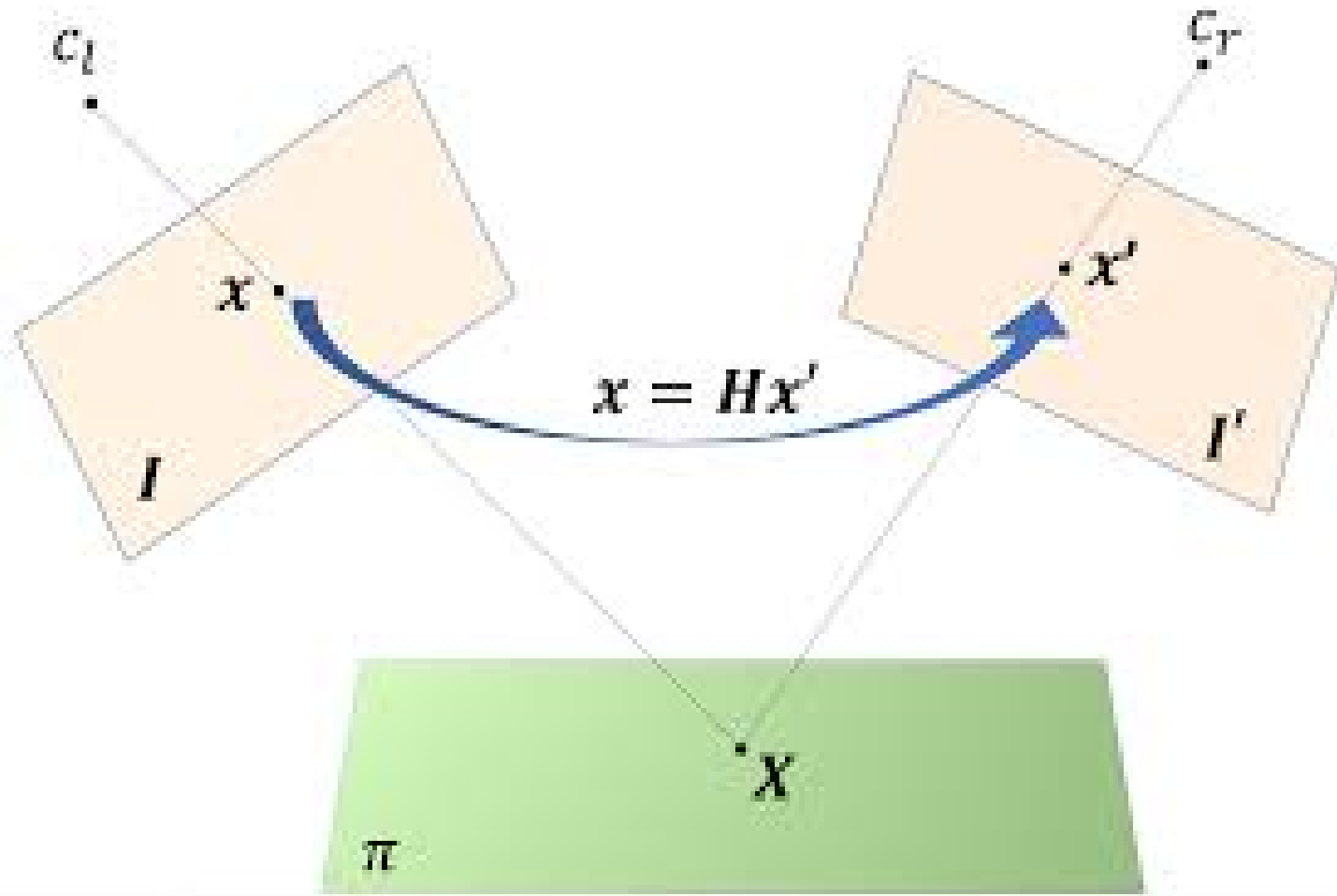
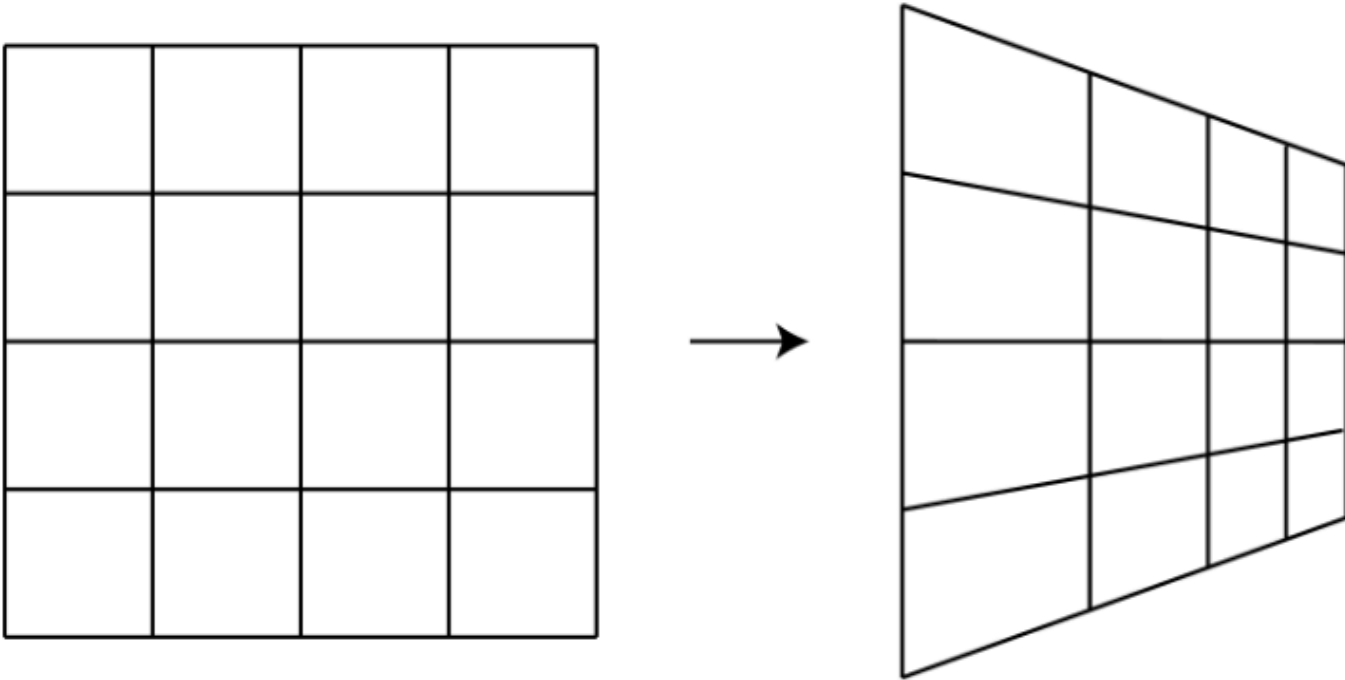
MẬT ĐỘ QUY ĐỔI TÀI TRỌNG (PCE-AWARE DENSITY)

Homography Matrix (Ma trận biến đổi phối cảnh):

Công thức Biểu diễn Ma trận

$$p' = Hp$$

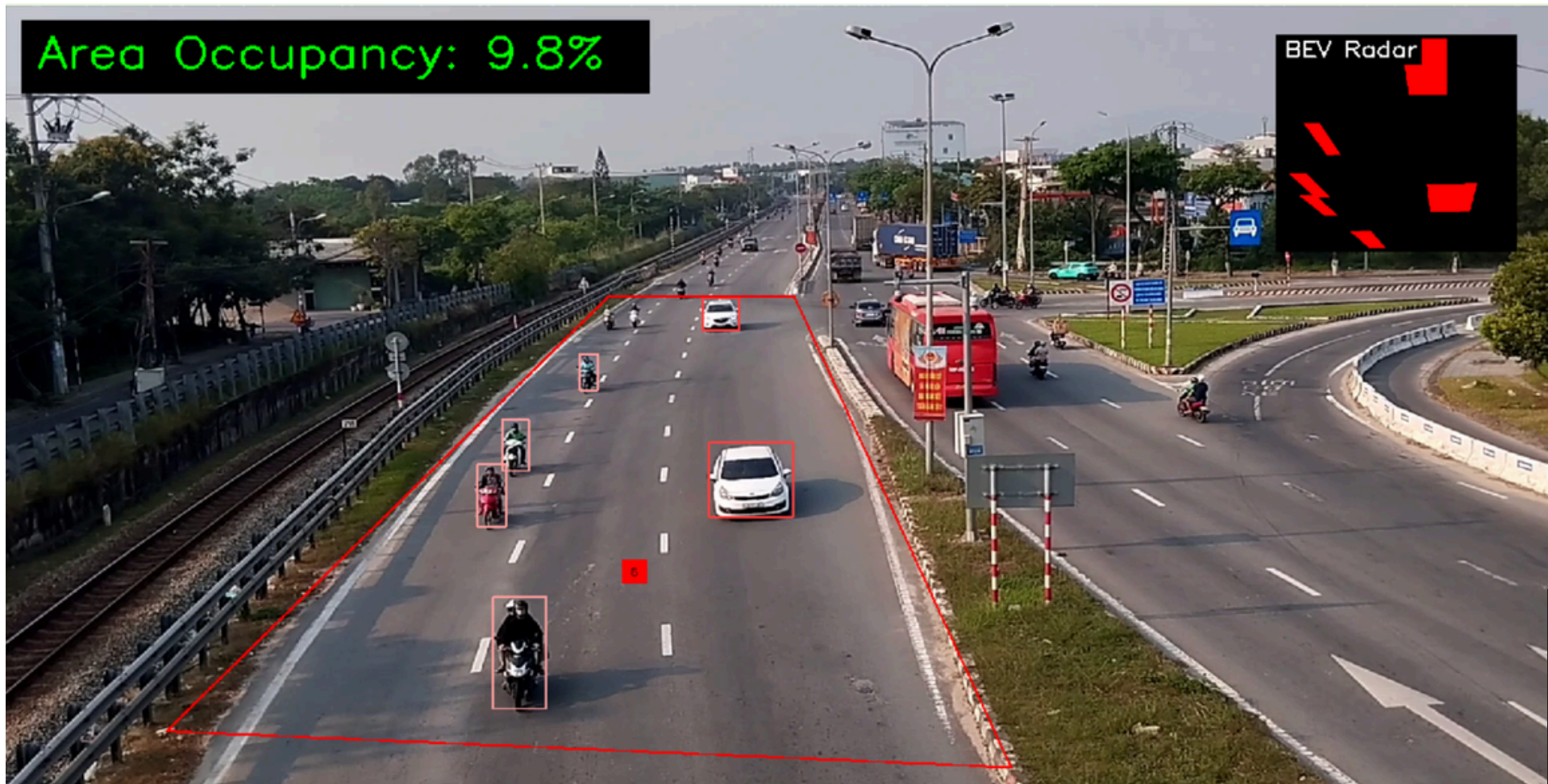
$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$



TỶ LỆ CHIẾM DỤNG DIỆN TÍCH QUA BẢN ĐỒ 2D (BEV AREA OCCUPANCY)

- Vấn đề cốt lõi:** Các phương pháp trước bị lỗi phối cảnh (Perspective Distortion): xe gần trông rất to, xe xa trông rất nhỏ, và xe lớn che lấp xe bé (Occlusion).
- Công nghệ cốt lõi:** Áp dụng **Ma trận Homography**(ma trận biến đổi phối cảnh) để trải phẳng đa giác ROI thành góc nhìn từ trên trời nhìn xuống (Bird-eye View).
- Cách tính toán:** Chuyển các Bounding Box thành mặt nạ điểm ảnh (Boolean Mask).

$$Area\ Occupancy\ (\%) = \frac{\text{Tổng điểm ảnh xe chiếm dụng trên 2D}}{\text{Tổng diện tích bản đồ 2D}} \times 100$$



```
SRC_ROI_VERTICES = np.array([
    [750, 400],    # Điểm 1: Góc trái trên
    [980, 400],    # Điểm 2: Góc phải trên
    [1250, 1050],  # Điểm 3: Góc phải dưới (mở rộng ra do góc phối cảnh)
    [200, 1000]   # Điểm 4: Góc trái dưới
], dtype=np.float32)
```

```
BEV_WIDTH = 500
```

```
BEV_HEIGHT = 500
```

```
DST_BEV_VERTICES = np.array([
    [0, 0],
    [BEV_WIDTH, 0],
    [BEV_WIDTH, BEV_HEIGHT],
    [0, BEV_HEIGHT]
], dtype=np.float32)
```

```
PERSPECTIVE_MATRIX = cv2.getPerspectiveTransform(SRC_ROI_VERTICES, DST_BEV_VERTICES)
TOTAL_BEV_PIXELS = BEV_WIDTH * BEV_HEIGHT
```

```
# Trích xuất 4 góc của Bounding Box trên camera
box_corners = np.array([
    [[x1, y1], [x2, y1], [x2, y2], [x1, y2]]
], dtype=np.float32)
```

```
# Dùng Ma trận Homography để bẻ Bounding Box này sang không gian BEV
transformed_corners = cv2.perspectiveTransform(box_corners, PERSPECTIVE_MATRIX)
```

```
# Vẽ hình đa giác vừa biến đổi lên mặt đường BEV bằng màu trắng (255)
# Kỹ thuật vẽ chồng này tự động loại bỏ lỗi tính trùng diện tích khi xe che khuất nhau
cv2.fillPoly(bev_canvas, bev_polygon, 255)
```

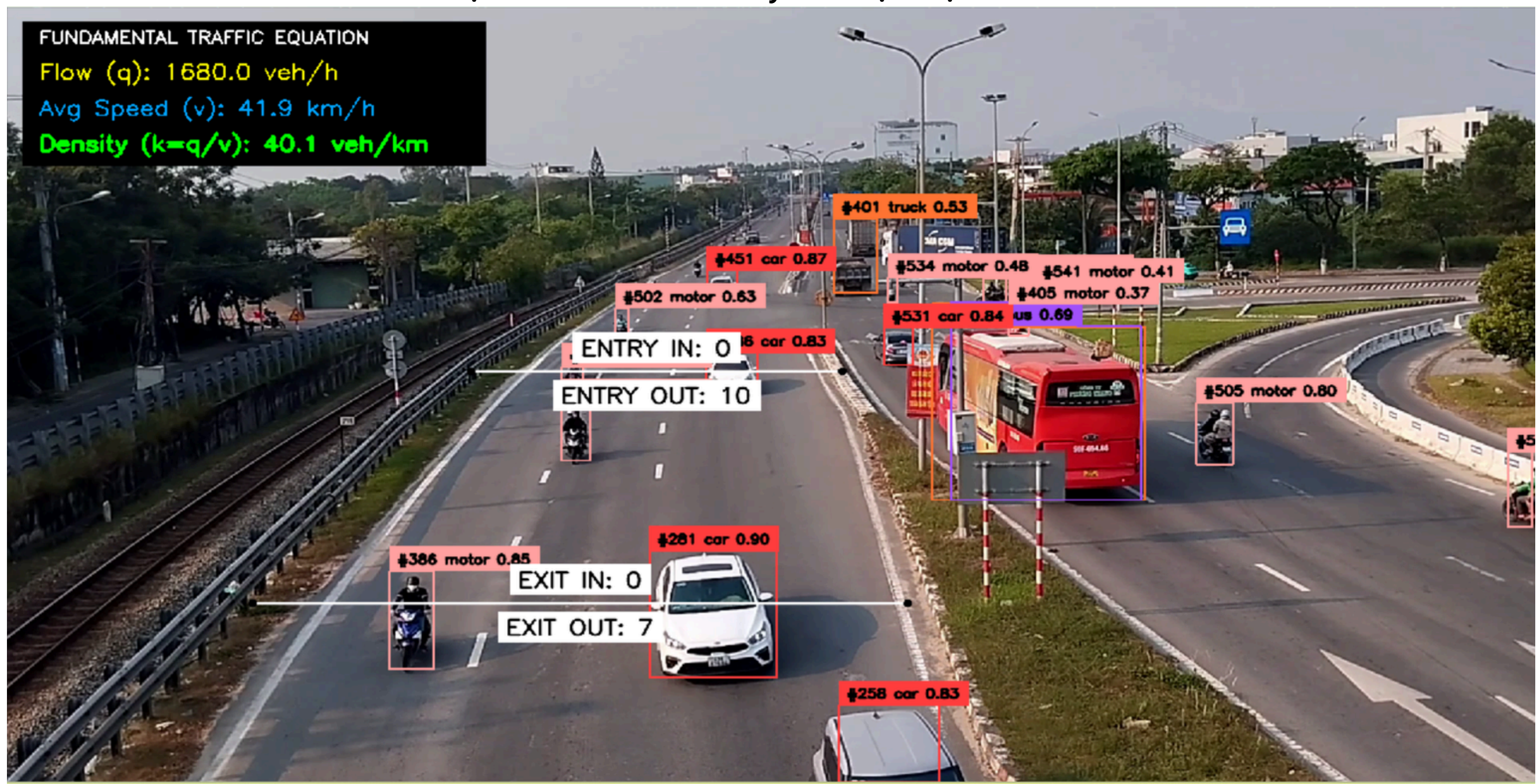
LÝ THUYẾT DÒNG GIAO THÔNG (FUNDAMENTAL TRAFFIC EQUATION)

Cách làm: Dựa vào mối quan hệ vĩ mô giữa Lưu lượng (q), Mật độ (k) và Vận tốc trung bình (v)

Công thức:

$$q = k \times v \implies k = \frac{q}{v}$$

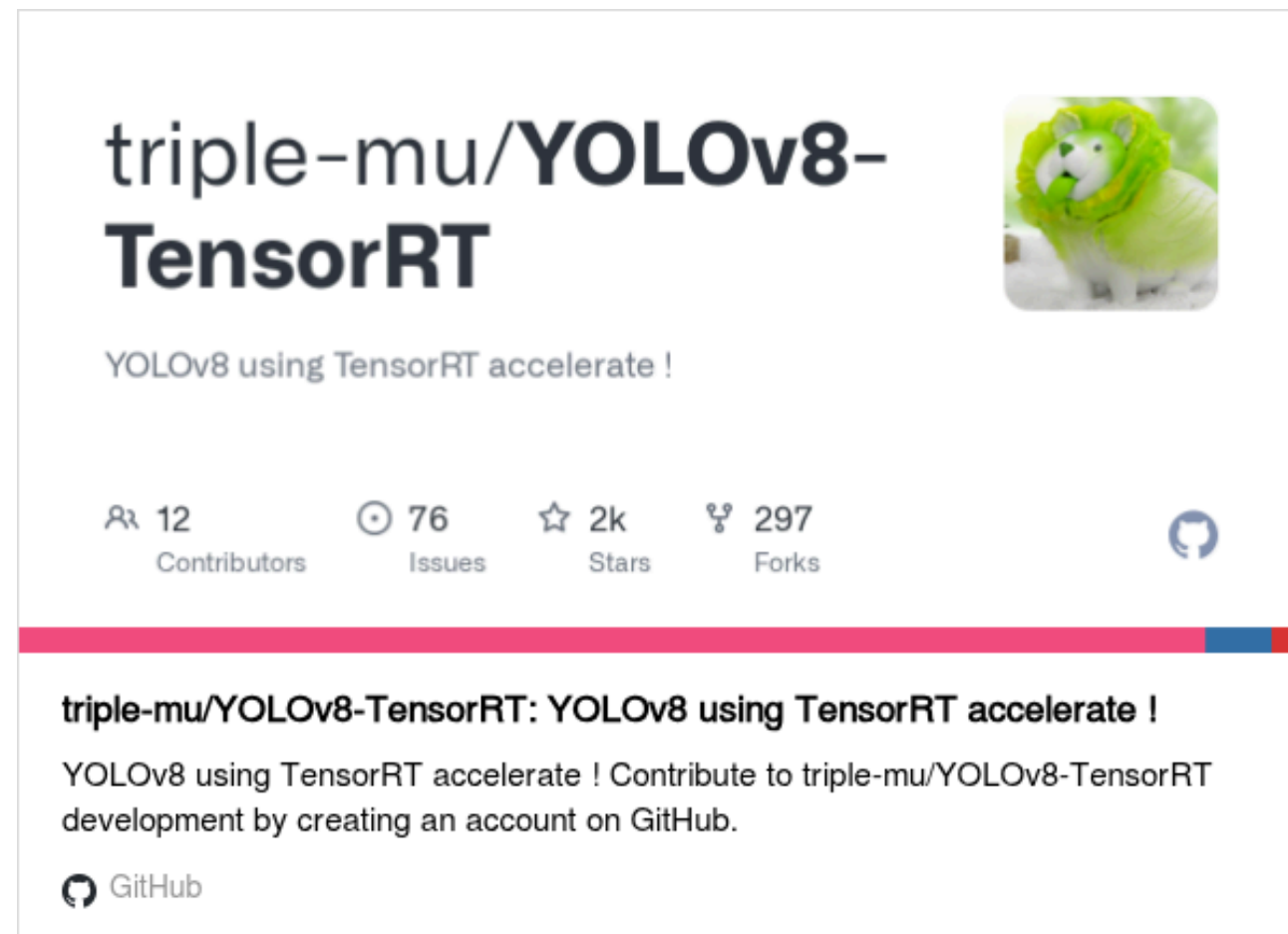
Cách triển khai: Đếm lưu lượng q (như đã làm ở phần đếm xe). Sau đó, đo thời gian một chiếc xe đi từ điểm A đến điểm B trên camera để tính vận tốc v. Từ đó suy ra mật độ k.



TRIỂN KHAI PIPELINE LÊN JETSON NANO BẰNG NVIDIA DEEPSTREAM SDK

- Edge Device không có CPU/GPU "khủng" như máy Server
- Pipeline code Python truyền thống tốn tài nguyên sao chép dữ liệu giữa CPU (RAM) và GPU (VRAM)
- Rớt Frame thê thảm, độ trễ giật lag hoàn toàn mất đi khả năng Real-time

GIẢI PHÁP




```
# --- Paths ---
WORK_DIR="${WORK_DIR:-$HOME/deepstream_yolo}"
DEEPSTREAM_YOLO_DIR="${DEEPSTREAM_YOLO_DIR:-$HOME/DeepStream-Yolo}"
CUDA_VER="${CUDA_VER:-10.2}"

# --- RTSP ---
LAPTOP_RTSP_URI="${LAPTOP_RTSP_URI:-}"
JETSON_RTSP_PORT="${JETSON_RTSP_PORT:-8555}"
JETSON_UDP_PORT="${JETSON_UDP_PORT:-5400}"
JETSON_RTSP_MOUNT="${JETSON_RTSP_MOUNT:-/ds-test}"

# --- Inference ---
PRE_CLUSTER_THRESHOLD="${PRE_CLUSTER_THRESHOLD:-0.25}"
NMS_IU_THRESHOLD="${NMS_IU_THRESHOLD:-0.45}"
# interval=N → inference every (N+1)th frame. Tracker fills gaps.
# Reduced to 1 (run AI every 2nd frame) to improve accuracy of tracking!
INFER_INTERVAL="${INFER_INTERVAL:-1}"

# --- Video Quality ---
# Reverted to 640x480 to allow NvDCF tracker to run at 30 FPS without freezi
STREAMMUX_WIDTH="${STREAMMUX_WIDTH:-640}"
STREAMMUX_HEIGHT="${STREAMMUX_HEIGHT:-480}"
OUTPUT_BITRATE="${OUTPUT_BITRATE:-2000000}"

# --- Tracker Settings ---
# NvDCF handles occlusions and stops flickering, making line-crossing work a
TRACKER_TYPE="${TRACKER_TYPE:-nvdcf}"
# NvDCF requires width and height to be multiples of 32 (only used if TRACKER
TRACKER_WIDTH="${TRACKER_WIDTH:-640}"
TRACKER_HEIGHT="${TRACKER_HEIGHT:-384}"
```

```
# --- Analytics: line-crossing vehicle counter ---
ENABLE_ANALYTICS="${ENABLE_ANALYTICS:-1}"
# Line crossing: Count vehicles moving DOWN (towards camera) on the main road
# Moved line LOWER (Y=420) where cars are significantly larger and bounding boxes are stable!
LC_X1="${LC_X1:-0}"
LC_Y1="${LC_Y1:-420}"
LC_X2="${LC_X2:-500}"
LC_Y2="${LC_Y2:-420}"
# Arrow: pointing DOWNwards
LC_DX1="${LC_DX1:-250}"
LC_DY1="${LC_DY1:-380}"
LC_DX2="${LC_DX2:-250}"
LC_DY2="${LC_DY2:-460}"

# --- Control ---
RUN_PIPELINE="${RUN_PIPELINE:-1}"
SKIP_SOURCE_CHECK="${SKIP_SOURCE_CHECK:-0}"

# =====

ONNX_FILE="${WORK_DIR}/best_deepstream.onnx"
LABELS_FILE="${WORK_DIR}/labels.txt"
CUSTOM_LIB="${WORK_DIR}/libnvdsinfer_custom_impl_Yolo.so"
INFER_CFG="${WORK_DIR}/config_infer_primary_yolov8.txt"
APP_CFG="${WORK_DIR}/deepstream_app_yolov8_rtsp.txt"
APP_CFG+="
label-lib-path=${LABELS_FILE}
batch-size=1
network-mode=2
num-detected-classes=4
interval=${INFER_INTERVAL}
gie-unique-id=1
process-mode=1
network-type=0
cluster-mode=2
maintain-aspect-ratio=1
parse-bbox-func-name=NvDsInferParseYolo
custom-lib-path=${CUSTOM_LIB}

## Class 0: bus
[class-attrs-0]
pre-cluster-threshold=0.25
nms-iou-threshold=${NMS_IU_THRESHOLD}
topk=50
border-color=0.0;1.0;0.0;1.0
```

bluenviron/ mediamtx

Ready-to-use SRT / WebRTC / RTSP / RTMP / LL-HLS / MPEG-TS / RTP media server and media proxy that...

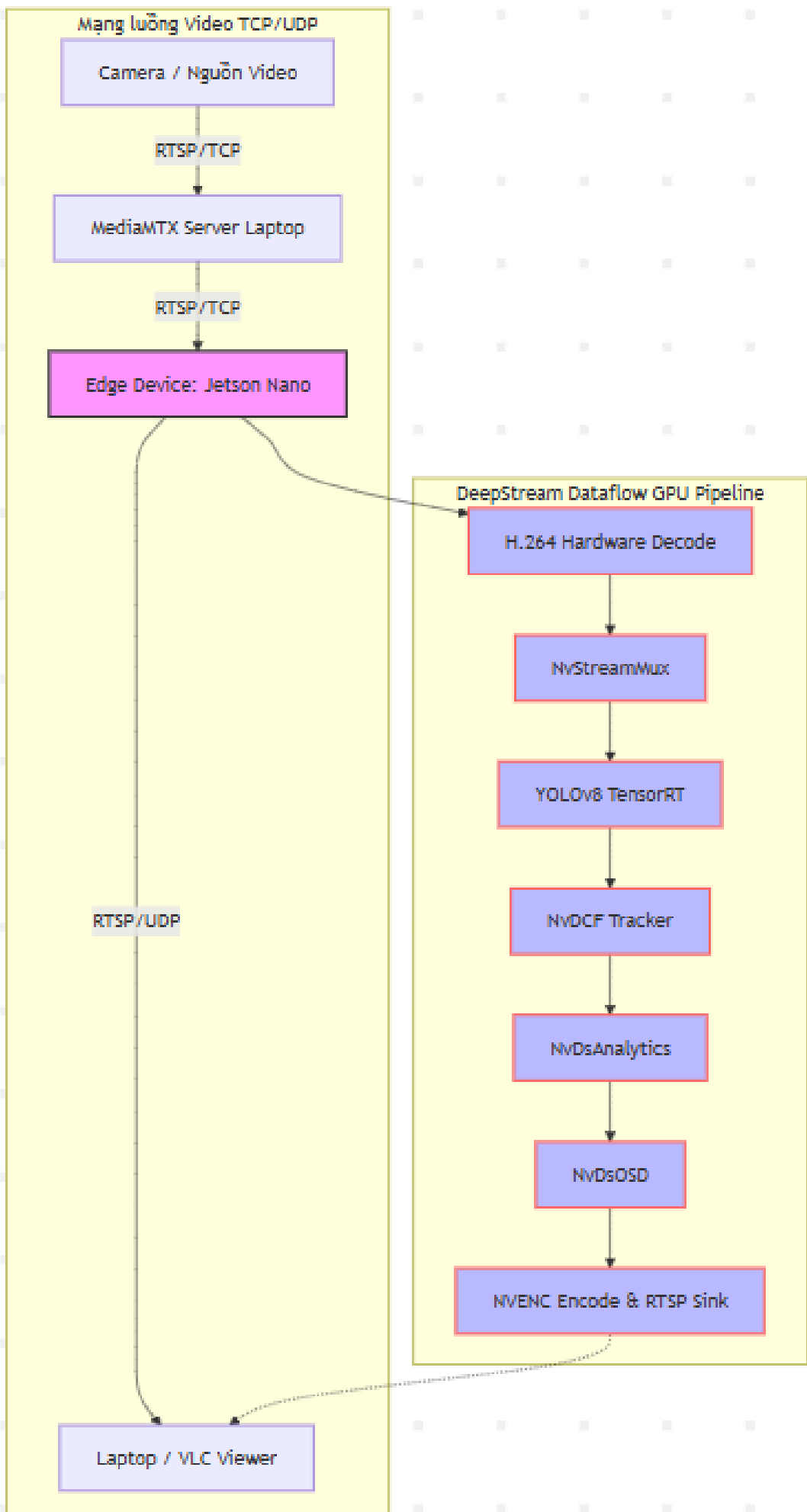
117
Contributors

184
Issues

1k
Discussions

18k
Stars

2k
Forks



Giải mã dữ liệu

Tiền xử lý dữ liệu

Suy luận

Tracking

Phân tích chuyên sâu

Vẽ boundingbox,text

Mã hóa dữ liệu và truyền dữ liệu đi

MỘT SỐ HÌNH ẢNH VÀ KẾT QUẢ THỬ NGHIỆM

```
(deepstream-app:54): GLib-GObject-WARN
**PERF:  0.00 (0.00)
reference in DPB was never decoded
**PERF:  0.00 (0.00)
H264: Profile = 66, Level = 0
NVMEDIA_ENC: bBlitMode is set to TRUE
**PERF:  41.80 (41.80)
**PERF:  30.05 (35.65)
**PERF:  30.02 (33.64)
**PERF:  30.20 (32.71)
**PERF:  29.37 (32.12)
**PERF:  30.45 (31.80)
**PERF:  29.95 (31.56)
**PERF:  30.15 (31.34)
**PERF:  29.81 (31.21)
**PERF:  29.93 (31.07)
**PERF:  30.05 (30.99)
**PERF:  30.00 (30.91)
**PERF:  29.99 (30.84)
**PERF:  30.24 (30.79)
**PERF:  29.95 (30.71)
**PERF:  30.05 (30.69)
**PERF:  29.99 (30.65)
```

Administrator: Windows Pow

Administrator: Windows Po

Administrator: Windows Pow

Administrator: Windows Pow

+

▼

-

□

26/04/14 07:06:42 INF [RTSP] [session 062c8061] destroyed: terminated

26/04/14 07:10:16 INF [RTSP] [conn [::1]:52929] opened

26/04/14 07:10:16 INF [RTSP] [session 219a9fa2] created by [::1]:52929

26/04/14 07:10:16 INF [path mystream] stream is available and online, 1 track (H264)

26/04/14 07:10:16 INF [RTSP] [session 219a9fa2] is publishing to path 'mystream'

26/04/14 07:10:16 INF [path mystream] RTP packets are too big (1460 > 1440), remuxing them into smaller ones

26/04/14 07:10:17 INF [RTSP] [conn 192.168.1.10:44048] opened

26/04/14 07:10:17 INF [RTSP] [session 9d1987ba] created by 192.168.1.10:44048

26/04/14 07:10:18 INF [RTSP] [session 9d1987ba] is reading from path 'mystream', with TCP, 1 track (H264)

26/04/14 07:11:55 INF [RTSP] [session 9d1987ba] destroyed: torn down by 192.168.1.10:44048

26/04/14 07:11:55 INF [RTSP] [conn 192.168.1.10:44048] closed: EOF

26/04/14 07:15:02 INF [RTSP] [conn 192.168.1.10:44056] opened

26/04/14 07:15:02 INF [RTSP] [session 927407d3] created by 192.168.1.10:44056

26/04/14 07:15:03 INF [RTSP] [session 927407d3] is reading from path 'mystream', with TCP, 1 track (H264)

26/04/14 07:17:37 INF [RTSP] [session 927407d3] destroyed: torn down by 192.168.1.10:44056

26/04/14 07:17:37 INF [RTSP] [conn 192.168.1.10:44056] closed: EOF

26/04/14 07:18:20 INF [RTSP] [conn 192.168.1.10:44060] opened

26/04/14 07:18:20 INF [RTSP] [session 166b2a8c] created by 192.168.1.10:44060

26/04/14 07:18:21 INF [RTSP] [session 166b2a8c] is reading from path 'mystream', with TCP, 1 track (H264)

26/04/14 07:21:01 INF [RTSP] [session 166b2a8c] destroyed: torn down by 192.168.1.10:44060

26/04/14 07:21:01 INF [RTSP] [conn 192.168.1.10:44060] closed: EOF

26/04/14 07:22:57 INF [RTSP] [conn 192.168.1.10:44068] opened

GPU [81.8%] 921MHz

Dsk [26.2G/58.9G]

PID	USER	GPU	TYPE	PRI	S	CPU%	MEM	[GPU MEM]	Command
20104	root	I	G	20	S	51.5	419M	414M	deepstream-app
5242	root	I	G	20	S	0.0	4.5M	76k	nvargus-daemon

[HW engines]	[Sensor]	[Temp]	[Power]	[Inst]	[Avg]
APE: 25.5MHz	AO	66.50C	POM 5V CPU	1.4W	1.4W
NVENC: 716MHzNVDEC: 716MHz	CPU	57.00C	POM 5V GPU	2.2W	2.1W
NVJPG: [OFF] SE: [OFF]	GPU	55.50C	POM_5V_IN	6.9W	6.6W
	PLL	54.00C			
	thermal	56.50C			

1ALL2GPU3CPU4MEM5ENG6CTRL7INFOQuit

(c) 2024, RE

A Survey of Lottery Ticket Hypothesis

Bohan Liu¹, Zijie Zhang², Peixiong He³, Zhensen Wang², Yang Xiao⁴, Ruimeng Ye⁵,
Yang Zhou³, Wei-Shinn Ku³, Bo Hui^{6*}

Carnegie Mellon University¹, University of Texas at San Antonio², Auburn University³,
Nankai University⁴, Australian National University⁵, University of Tulsa⁶

Abstract

The Lottery Ticket Hypothesis (LTH) states that a dense neural network model contains a highly sparse subnetwork (i.e., winning tickets) that can achieve even better performance than the original model when trained in isolation. While LTH has been proved both empirically and theoretically in many works, there still are some open issues, such as efficiency and scalability, to be addressed. Also, the lack of open-source frameworks and consensual experimental setting poses a challenge to future research on LTH. We, for the first time, examine previous research and studies on LTH from different perspectives. We also discuss issues in existing works and list potential directions for further exploration. This survey aims to provide an in-depth look at the state of LTH and develop a duly maintained platform to conduct experiments and compare with the most updated baselines.

Unstructured Pruning (Cắt tỉa không cấu trúc)

- Cách làm: Xóa các trọng số đơn lẻ w xấp xỉ 0 dựa trên độ lớn (Magnitude-based).
- Ưu điểm: Giữ được độ chính xác rất tốt.
- Nhược điểm: Khó tăng tốc trên phần cứng thông thường (CPU/GPU) vì ma trận thưa cần thư viện hỗ trợ đặc biệt.

Structured Pruning (Cắt tỉa có cấu trúc)

- Cách làm: Xóa nguyên cả một cụm nơ-ron, một hàng trong ma trận, hoặc cả một Channel (kênh) trong CNN.
- Ưu điểm: Giảm kích thước file model và tăng tốc độ suy luận (Inference) ngay lập tức trên mọi phần cứng.
- Nhược điểm: Dễ làm giảm độ chính xác hơn so với Unstructured.

```

# -----
# EB Observer
# -----

class EB_Observer:
    def __init__(self, threshold=0.98):
        self.prev_mask = None
        self.threshold = threshold
        self.stable_epochs = 0

    def check_stability(self, model):
        gammas = []
        for m in model.modules():
            if isinstance(m, torch.nn.BatchNorm2d):
                gammas.append(m.weight.data.abs().cpu().numpy())

        if not gammas: # FIX 1: tránh crash nếu không tìm thấy BN layer
            return False

        all_gammas = np.concatenate(gammas)
        median = np.median(all_gammas)
        current_mask = all_gammas > median

        if self.prev_mask is not None:
            similarity = np.mean(current_mask == self.prev_mask)
            epoch = observer.stable_epochs
            print(f" [EB] Cấu trúc ổn định: {similarity*100:.2f}% | streak: {self.stable_epochs}/5")

            if similarity > self.threshold:
                self.stable_epochs += 1
            else:
                self.stable_epochs = 0 # FIX 2: reset dừng chỗ (code gốc reset sau khi đã check)

        self.prev_mask = current_mask
        return self.stable_epochs >= 12

# -----
# Callback
# -----

observer = EB_Observer(threshold=0.995)

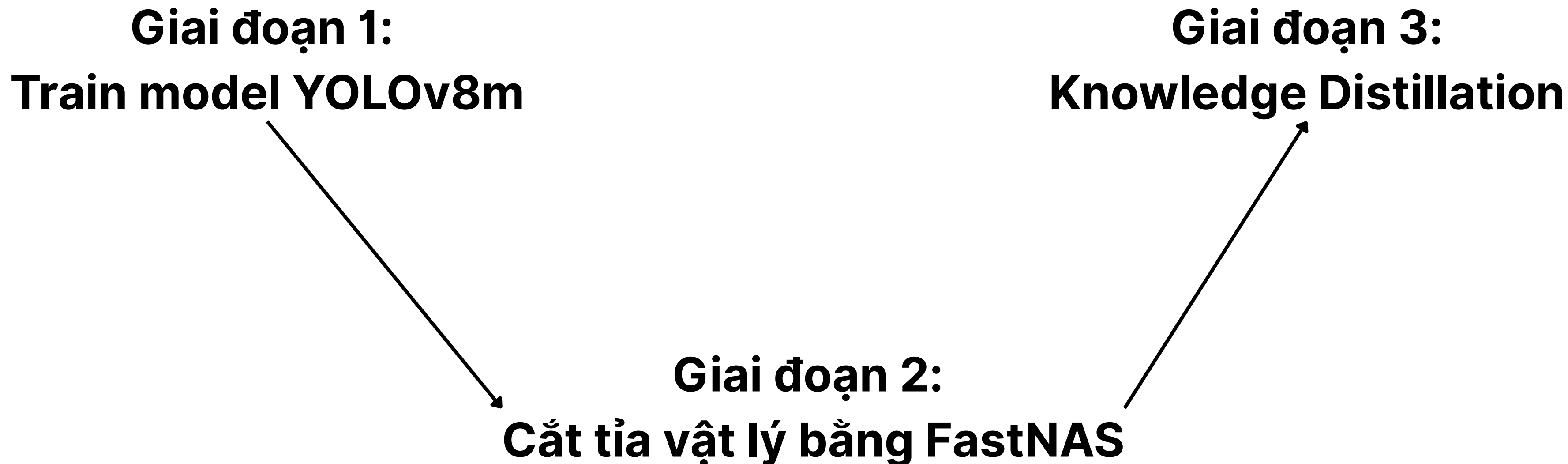
def on_train_epoch_end(trainer):
    is_stable = observer.check_stability(trainer.model)
    if is_stable:
        print("\n" + "="*60)
        print("!!! ĐÃ TÌM THẤY EB-TICKET: Cấu trúc đã ổn định !!!")

```

EB TICKETS

- Early-bird Tickets chính là các mạng con (sub-networks) có cấu trúc quan trọng được phát hiện ở giai đoạn đầu của quá trình huấn luyện.
- Ý tưởng: Chỉ cần train mô hình khoảng 10% - 30% tổng thời gian.
- Phát hiện: Quan sát sự thay đổi của cấu trúc mạng (thường qua các tham số của lớp BatchNorm). Khi cấu trúc này ngừng thay đổi đáng kể, bạn đã có Early-bird Ticket.
- Hành động: Dừng train toàn bộ mô hình, cắt tỉa ngay lúc đó và chỉ tập trung train "mạng con" vừa tìm được.

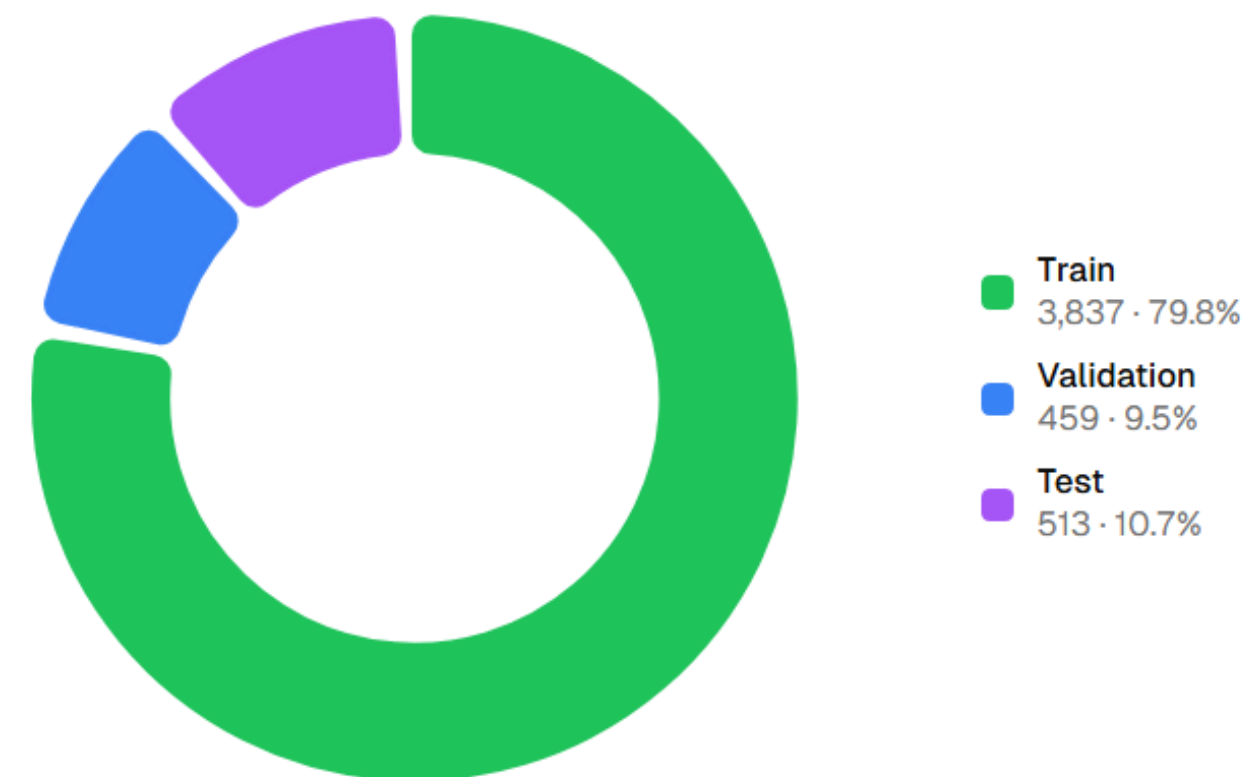
Giải pháp tối ưu hóa YOLOv8n bằng FastNAS & KD



```
#  
# Train  
#  
model = YOLO('yolov8n.pt')  
model.add_callback('on_train_epoch_end', on_train_epoch_end)  
  
model.train(  
    data='/content/drive/MyDrive/Work3.yolov8/data.yaml',  
    epochs=150,  
    imgsz=640,  
    batch=32,  
    optimizer='AdamW',  
    lr0=0.001,  
  
    box=10.0,  
    cls=2.0,  
    dfl=2.0,  
  
    # Đã cover tĩnh  
    fliplr=0.0,  
    scale=0.0,  
    hsv_s=0.0,  
    hsv_v=0.0,  
    erasing=0.0,  
  
    # Chưa cover  
    hsv_h=0.015,  
    degrees=5.0,  
    shear=2.0,  
    perspective=0.0003,  
    translate=0.1,  
    mosaic=1.0,  
    mixup=0.1,  
    close_mosaic=20,  
    copy_paste=0.3,  
  
    cos_lr=True,  
    device=0,  
    project='/content/drive/MyDrive/PBL5_Final_Results',  
    name='v8m_eb_observer'  
)
```

Split Distribution

4,809 images · 100.0% labeled



Top Classes

4 classes · 111,193 annotations



Top Classes

4 classes · 111,193 annotations



- motor
55,550 · 50.0%
- car
46,645 · 41.9%
- truck
5,767 · 5.2%
- bus
3,231 · 2.9%

Split Distribution

4,809 images · 100.0% labeled



- Train
3,837 · 79.8%
- Validation
459 · 9.5%
- Test
513 · 10.7%

Kết quả train YOLOv8m

Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	17/17	1.3s/it	21.9s
all	513	12976	0.893	0.832	0.896	0.692			
bus	216	387	0.93	0.827	0.922	0.765			
car	509	5289	0.925	0.858	0.929	0.717			
motor	494	6670	0.914	0.857	0.916	0.68			
truck	267	630	0.804	0.786	0.82	0.607			

Speed: 3.4ms preprocess, 25.3ms inference, 0.0ms loss, 2.7ms postprocess per image

Saving /content/runs/detect/val6/predictions.json...

Results saved to /content/runs/detect/val6

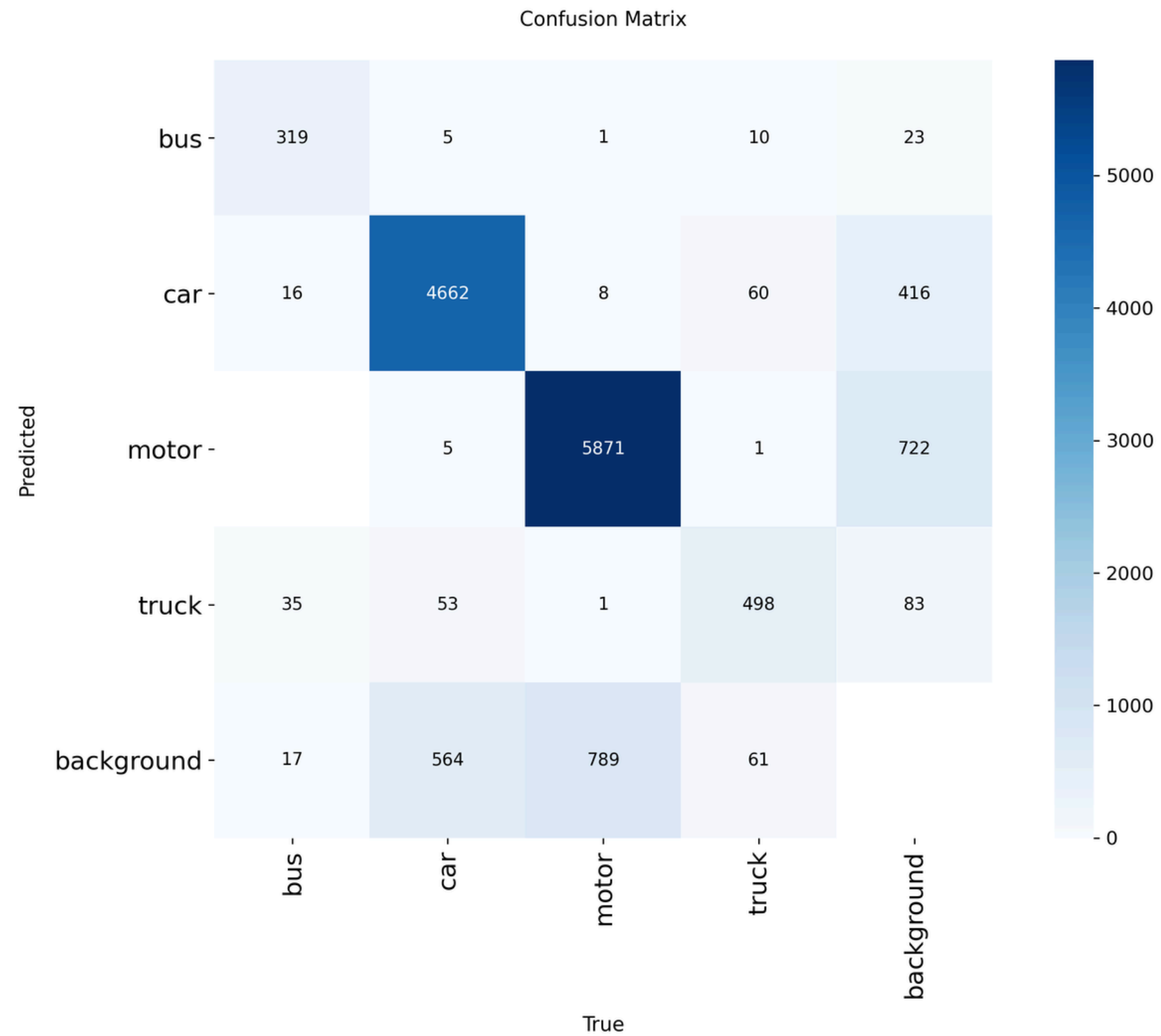
mAP50: 0.8964

mAP50-95: 0.6921

Precision: 0.8931

Recall: 0.8322

Kết quả train YOLOv8m



Phương pháp cắt tỉa vật lý YOLOv8n

Thay vì cắt tỉa thủ công, chúng ta sử dụng thuật toán FastNAS từ thư viện modelopt của NVIDIA:

- Cơ chế: Thuật toán thực hiện tìm kiếm cấu trúc mạng tối ưu (Neural Architecture Search) dựa trên ràng buộc phần cứng.
- Ràng buộc (Constraints): Giảm 70% FLOPs so với mô hình gốc.
- Hàm mục tiêu (Score Function): Sử dụng chỉ số fitness (tổng hợp của mAP50 và mAP50-95) làm thước đo để quyết định lớp nào nên giữ, lớp nào nên cắt.

Knowledge Distillation

Sau khi bị cắt tỉa 70%, mô hình Student sẽ bị sụt giảm kiến thức nghiêm trọng. Chúng ta sử dụng một mô hình Teacher mạnh hơn (YOLOv8 Medium đã train tốt) để dẫn dắt:

$$L_{total} = L_{YOLO} + \lambda \cdot L_{KD}$$

Knowledge Distillation

Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	17/17 1.3s/it 21.3s
all	513	12976	0.863	0.79	0.868	0.63	
bus	216	387	0.867	0.822	0.896	0.725	
car	509	5289	0.909	0.817	0.9	0.638	
motor	494	6670	0.913	0.818	0.902	0.62	
truck	267	630	0.763	0.705	0.773	0.537	

Speed: 2.3ms preprocess, 3.9ms inference, 0.0ms loss, 1.7ms postprocess per image

Saving /content/runs/detect/val/predictions.json...

Results saved to /content/runs/detect/val

mAP50: 0.8679

mAP50-95: 0.6301

Precision: 0.8630

Recall: 0.7903

Knowledge Distillation

Yolov8m

mAP50: 0.8964

mAP50-95: 0.6921

Precision: 0.8931

Recall: 0.8322

Yolov8n đã prune

mAP50: 0.8679

mAP50-95: 0.6301

Precision: 0.8630

Recall: 0.7903

Knowledge Distillation

Yolov8n gốc

```
orig_model = YOLO('yolov8n.pt')
```

```
orig_model.info()
```

```
YOLOv8n summary: 129 layers, 3,157,200 parameters, 0 gradients, 8.9 GFLOPs  
(129, 3157200, 0, 8.8575488)
```

Yolov8n đã prune

```
pruned_model.info()
```

```
Model summary: 130 layers, 2,036,268 parameters, 2,036,268 gradients, 5.7 GFLOPs  
(130, 2036268, 2036268, 5.7181695999999995)
```